JOHANNES KEPLER
UNIVERSITÄT LINZ

257.130
Communications Engineering
(Kompetenztraining)
2023W

Lernaufwand:
75 Stunden/Person
(Zeitaufzeichnungen erforderlich)

SUGA
Smart Urban Gardening Assistent

Gruppe 03

Projektleiter:

12242893, Argjent Serifi, argjent.serifi@icloud.com

Teammitglieder:

11921201, Natalia Chifa

12241204, Valon Hyseni

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at
DVR 0093696

INHALTSVERZEICHNIS

1. Lernvertag 5
1.1. Projektbeschreibung 5
1.1.1. Projektidee 5
1.1.2. Theoretische Perspektive 5
1.1.3. Praktische Perspektive 5
1.2. Kompetenzdarstellung 6
1.2.1. Argjent Serifi 6
1.2.2. Natalia Chifa 6
1.2.3. Valon Hyseni 7
1.3. Ressourcen 7
1.4. Meilensteine 8
1.5. Ergebnissicherung 8
1.5.1. Arduino mit einlesenden Sensoren 8
1.5.2. Projektkonfiguration 9
1.5.3. Frontend 9
1.5.4. Datenkommunikation zwischen Embedded System und Webanwendung 9
1.5.5. Backend 9
1.5.6. Dokumentation 9
1.5.7. Smart Urban Gardening Assistent mit Zeitaufzeichnung und Projektportfolio 9
2. Theoretische Ausarbeitung 10
2.1. IoT – Internet of Things [1] 10
2.1.1. Kommunikation [2] 10
2.1.2. Architektur [3] 10
2.2. Embedded System 11
2.2.1. Arduino Uno [4, 5, 6, 7, 8] 11
2.2.2. DHT11 [9] 13
2.2.3. BH1750 [10] 13

2.2.4. ESP8266 [11, 12] 14
2.3. Datenkommunikation mittels HTTP [14, 15] 14
2.3.1. Methoden 14
2.3.2. Statuscodes 15
2.4. Webanwendung 16
2.4.1. Frontend 16
2.4.2. Backend 18
3. Praktische Ausarbeitung 20
3.1. Meilenstein 1 20
3.1.1. Temperatur und Luftfeuchtigkeit 20
3.1.2. Projekteinrichtung des Frontends 21
3.2. Meilenstein 2 23
3.2.1. Startseite 23
3.2.2. Hauptseite 29
3.2.3. Datenmodell 38
3.3. Meilenstein 3 41
3.3.1. Vollendung des Frontends 41
3.3.2. Lichtstärke 40
3.3.3. Projekteinrichtung des Backends 40
3.3.4. GET – Endpoints 40
3.3.5. POST – Endpoints 40
3.3.6. PUT – Endpoint 40
3.3.7. DELETE – Endpoints 40
3.4. Endabgabe 40
3.4.1. Kommunikation zwischen Webanwendung und Embedded System 40
4. Reflexion 40
4.1. Meilenstein 1 40
4.1.1. Argjent Serifi 40
4.1.2. Natalia Chifa 40

4.1.3. Valon Hyseni 40
4.2. Meilenstein 2 41
4.2.1. Argjent Serifi 41
4.2.2. Natalia Chifa 41
4.2.3. Valon Hyseni 42
4.3. Meilenstein 3 42
4.3.1. Argjent Serifi 42
4.3.2. Natalia Chifa 42
4.3.3. Valon Hyseni 43
4.4. Endabgabe 43

1. Lernvertag

1.1. Projektbeschreibung

1.1.1. Projektidee

SUGA (Smart Urban Gardening Assistent) ist eine innovative Lösung für PflanzenliebhaberInnen und FreizeitgärtnerInnen, um die Gesundheit ihrer Pflanzen in städtischen Umgebungen zu optimieren. AnwenderInnen können Pflanzen in der Webanwendung entweder über Desktop oder Smartphone hinzufügen und Sensoren messen kontinuierlich Umweltbedingungen wie Temperatur, Luftfeuchtigkeit und Lichtstärke. Infolgedessen werden passende Statusmeldungen und Empfehlungen angezeigt, um sicherzustellen, dass jede Pflanze in idealen Voraussetzungen wächst.

Tabelle 1: Beispiel anhand einer Tomate (ChatGPT, 2023)

	Temperatur	**Luftfeuchtigkeit**	**Lichtstärke**
Ideal	20°C - 24°C	50% - 70%	10.000 Lux – 20.000 Lux
Akzeptabel	18°C - 26°C	40% - 80%	8.000 Lux – 25.000 Lux
Mangelhaft	unter 18°C über 26°C	unter 40% über 80%	unter 8.000 Lux über 25.000 Lux

1.1.2. Theoretische Perspektive

Die theoretische Ausarbeitung beinhaltet alle integrierten Technologien, welche zur Implementierung vom Smart Urban Gardening Assistent notwendig sein werden.

Embedded System: Mikrocontroller (Arduino) und Sensoren (DHT11, BH1750 und ESP8266), um die Umweltbedingungen mit C++ zu erfassen und über HTTP (Hypertext Transfer Protocol) an die Webanwendung zu senden.

Webanwendung: Benutzer- und Pflanzeninformationen werden in einer relationalen Datenbank (mySQL) verwaltet. Implementierung des Frontends erfolgt mittels HTML (Hypertext Markup Language), CSS (Cascading Style Sheets, inklusive Tailwind CSS), JavaScript (unter Verwendung von Vue, Nuxt und weiteren kleinen Bibliotheken, welche im Laufe der Entwicklung hinzukommen).

1.1.3. Praktische Perspektive

Die praktische Ausarbeitung beinhaltet die Implementierung eines abgerundeten Systems. SUGA beginnt mit den Funktionen zur Anmeldung sowie Registrierung, um bei einem Gerätewechsel keine Daten, welche im Backend in der Datenbank verwaltet werden, zu verlieren. Nachfolgend bietet die Hauptseite eine Anzeige der eingelesenen Umweltbedingungen des Embedded System, eine Möglichkeit, um Pflanzen hinzuzufügen oder zu entfernen, eine Gesamtansicht aller Pflanzen mit Namen, wichtigen Informationen, und dem Status (ideal, akzeptabel, mangelhaft). Damit sich das System abgerundet anfühlt, werden ein responsive Design, eine Sprachauswahl, Such- sowie Filtermöglichkeiten und weitere kleine Funktionalitäten ebenfalls implementiert.

1.2. Kompetenzdarstellung

1.2.1. Argjent Serifi

Kenntnisse im Fachbereich der Embedded Systems und Implementierung von Frontends für Webanwendungen liegen aufgrund schulischer Ausbildung an der Höheren Technischen Bundeslehranstalt mit der Fachrichtung Informationstechnologie sowie kurzer Berufserfahrung als Softwareentwickler vor. Bislang ist dieses Wissen jedoch auf ihre jeweiligen Themengebiete beschränkt geblieben und wurde in der Konstellation noch nie in einem integrierten System zusammengeführt. Das Projekt soll das Verständnis für den gesamten Prozess der Datenverarbeitung, beginnend bei der Datenerfassung mittels Sensoren bis hin zur Darstellung in der Webanwendung, erweitern. Besonders die Herausforderungen der Datenkommunikation, inklusive der Auswahl und Implementierung geeigneter Übertragungsprotokolle, sowie die effiziente Datenverwaltung im Backend verursachen bislang bestehende Schwierigkeiten. Das anstehende Projekt bietet die Gelegenheit, diese Wissensdefizite zu überwinden und ein umfassenderes Verständnis für die Entwicklung und Integration von Embedded Systemen und Webanwendungen zu erlangen. Durch die praktische Anwendung dieser Technologien innerhalb des Projekts wird eine erhebliche Verbesserung der Fähigkeiten im Kontext eines abgerundeten Systems erwartet.

1.2.2. Natalia Chifa

Die bisherigen Erfahrungen und Kenntnisse im Bereich Embedded Systems sind überwiegend von theoretischer Natur, erworben während des Studiums der Wirtschaftsinformatik. Praktische Fähigkeiten in der Umsetzung von Embedded Systems fehlen bislang. Das vorliegende Projekt eröffnet die Möglichkeit, theoretisches Wissen in die Praxis umzusetzen und Fähigkeiten im Bereich der Embedded Systems zu erweitern. In der aktuellen beruflichen Position in der SAP-Abteilung ist die Verantwortung die Verwaltung eines Webshops, welcher Daten aus dem SAP-System abruft und an Kunden verkauft. Dabei wurden Erfahrungen im Umgang mit webbasierten Technologien gesammelt, insbesondere bei der Anpassung von Texten und Inhalten. Beispielsweise erfolgte die Bearbeitung von Texten auf der Willkommensseite des Webshops und die Anpassung von Textschlüsseln mittels HTML in einem Editor. Obwohl die Erfahrung im Gebiet der Webanwendungen begrenzt ist, wurde ein grundlegendes Verständnis von HTML und Webseitengestaltung erlangt. Ebenso wurde mit Datenbanken wie MySQL gearbeitet, insbesondere im Rahmen der Datenmodellierung und des Data Knowledge Engineering während des Studiums. Dabei erfolgte die Arbeit mit SQLDeveloper zur Erstellung und Verwaltung von Datenbanken. Was die Programmierung von Mikrocontrollern wie dem Arduino betrifft, wurden bisher keine praktischen Kompetenzen gesammelt. Dennoch besteht eine hohe Motivation, diese Fertigkeiten im Rahmen dieses Projekts zu erlernen und anzuwenden. Das Ziel des Projekts ist es, praktische Erfahrungen in der Programmierung von Mikrocontrollern zu sammeln und Kenntnisse in diesem Bereich zu vertiefen.

1.2.3. Valon Hyseni

Bereits vorhandene Erfahrungen mit Mikrocontrollern, insbesondere dem Arduino, ermöglichen es, Hardwarekomponenten, wie Sensoren und Mikrocontroller, korrekt zu verkabeln sowie konfigurieren und Methoden zu programmieren, welche die Umweltbedingungen der Pflanzen zuverlässig messen. Diese technische Kompetenz aus der schulischen Ausbildung an der Höheren Technischen Bundeslehranstalt mit der Fachrichtung Informationstechnologie bildet eine solide Grundlage für die Datenerfassung und -übertragung. Allerdings liegt die größte Herausforderung in Umsetzung einer funktionsfähigen Webanwendung, für die grafische Darstellung der gemessenen Daten. Die Realisierung einer robusten und benutzerfreundlichen Webseite erfordert ein tiefes Verständnis von Technologien wie HTML, CSS, JavaScript und Datenbanken wie MySQL, mit welchen es bisher nur wenig, bis gar keine Berührungspunkte gab. Zusätzlich sind bislang keine Kenntnisse über Frameworks und Bibliotheken wie Vue und Nuxt existent, um das Frontend der Anwendung effizient zu gestalten. Die Realisierung dieses Projekts soll dabei helfen, praktische Erfahrungen zu sammeln, bereits vorhandene Fähigkeiten zu festigen und Wissen für bisher nicht bekannte Themen anzueignen. Wichtig ist es, alle Komponenten eines gesamten Systems abzudecken und somit umfangreiche Fähigkeiten zu erarbeiten.

1.3. Ressourcen

Embedded System:

- Arduino, Arduino IDE und C++
- DHT11, BH1750 und ESP8266
- HTTP (Hypertext Transfer Protocol)

Webanwendung:

- Visual Studio Code und mySQL
- HTML (Hypertext Markup Language), CSS (Cascading Style Sheets) und JavaScript
- Node, Vue, Nuxt, Tailwind CSS und weitere weiteren kleinen Frameworks, Komponenten und Bibliotheken, welche im Laufe der Entwicklung hinzukommen

1.4. Meilensteine

Meilenstein #1 05. November 2023 (23:59)

Tabelle 2: Meilenstein #1

Titel:	**Theoretische Ausarbeitung und Einrichtung**
Deliverables:	Projektportfolio (theoretische Ausarbeitung)
	Zeitaufzeichnung (laufend)
	Arduino mit einlesenden Sensoren (Ausgabe in Konsole)
	Projektkonfiguration (Anzeige „hello world“)

Meilenstein #2 10. Dezember 2023 (23:59)

Tabelle 3: Meilenstein #2

Titel:	**Praktische Ausarbeitung**
Deliverables:	Projektportfolio (teilweise praktische Ausarbeitung)
	Zeitaufzeichnung (laufend)
	Datenkommunikation zwischen Embedded System und Webanwendung
	Frontend

Meilenstein #3 14. Januar 2024 (23:59)

Tabelle 4: Meilenstein #3

Titel:	**Projektportfolio und Smart Urban Gardening Assistent**
Deliverables:	Projektportfolio (beendet)
	Zeitaufzeichnung (beendet)
	Backend
	Dokumentation
	Smart Urban Gardening Assistent (beendet)

1.5. Ergebnissicherung

1.5.1. Arduino mit einlesenden Sensoren

Die physische Hardwarekomponente des Projekts, bestehend aus dem Mikrocontroller und den Sensoren, misst Umweltbedingungen und gibt diese in der Konsole aus. Somit wird die Grundlage für die Datenbeschaffung und Kommunikation mit der Webanwendung aufgebaut, und zugleich die Funktionalität des Mikrocontrollers sowie der Sensoren geprüft, um mögliche Defekte frühzeitig zu identifizieren und gegebenenfalls alternative Hardware zu beschaffen.

1.5.2. Projektkonfiguration

Hierbei wird der fundamentale Baustein für die Webanwendung etabliert und versichert, dass die Entwicklungsumgebung samt den eingesetzten Frameworks korrekt konfiguriert wurde. Darüber hinaus bietet es eine initiale Verzeichnisstruktur, an der sich alle Teammitglieder orientieren können, um die Entwicklung reibungslos fortzusetzen.

1.5.3. Frontend

Mit der Finalisierung des zweiten Meilensteins wird das Design eines Icons und einer Benutzeroberfläche inklusive Funktionalitäten unter Berücksichtigung der benötigten, flexibel anpassbaren Testdaten abgeschlossen. Die Implementierung des Backends erfolgt erst nachdem die Datenstruktur im Frontend festgelegt wurde, um den Entwicklungsprozess ohne Bedarf an späteren Überarbeitungen zu beschleunigen.

1.5.4. Datenkommunikation zwischen Embedded System und Webanwendung

Das zentrale Ziel des Projekts ist es eine kontinuierliche Übertragung, Verarbeitung und Darstellung der Umweltbedingungen in Echtzeit in der Webanwendung zu ermöglichen. Infolgedessen sollen Benutzer die Möglichkeit haben, die Gesundheit ihrer städtischen Pflanzen zu überwachen.

1.5.5. Backend

Eine Authentifizierung und Autorisierung der Benutzer ist notwendig, um demzufolge Benutzer- sowie Pflanzeninformationen, unabhängig vom verwendeten Gerät in einer konfigurierten Datenbank zu speichern und zugänglich zu machen.

1.5.6. Dokumentation

Protokollierungen tragen zur Verständlichkeit und Wartbarkeit des Codes bei, indem es erklärende Kommentare sowie Beschreibungen bereitstellt und den Wissenstransfer innerhalb der Teammitglieder erleichtert. Die klare Dokumentation im Code erleichtert sowohl das Debugging als auch die Fehlerbehebung und ist somit wichtig für die Effizienz, Qualität sowie Nachhaltigkeit, weswegen es zu einem abgerundeten Projekt dazugehört.

1.5.7. Smart Urban Gardening Assistent mit Zeitaufzeichnung und Projektportfolio

Dies kennzeichnet den Projektabschluss, bei dem das System in sich geschlossen und vollkommen funktionsfähig ist und sowohl Zeitaufzeichnung als auch Projektportfolio erfolgreich beendet wurden.

2. Theoretische Ausarbeitung

2.1. IoT – Internet of Things [1]

Das Konzept des Internet of Things (IoT) stellt eine technologische Innovation dar, um die Interkonnektivität physischer und virtueller Entitäten zu ermöglichen und die Kollaboration mittels Informations- und Kommunikationstechnologien zu fördern. Es referenziert Geräte, welche mit Sensoren, Rechenkapazitäten, Software oder sonstigen verwandten technologischen Elementen ausgestattet sind und sich mit anderen Geräten über das Internet oder alternative Kommunikationsnetzwerke verbinden, um Informationen auszutauschen. In einem systemischen Zusammenhang ermöglicht IoT die Echtzeitüberwachung und Regulierung von Umgebungsparametern, indem es physische Geräte vernetzt, um Daten zu akquirieren, zu transferieren und zu verarbeiten.

2.1.1. Kommunikation [2]

Die Datenkommunikation im IoT ist ein wichtiger Aspekt, um die reibungslose Übertragung von Informationen zwischen Entitäten zu gewährleisten. Die Auswahl des geeigneten Protokolls ist essenziell, weil es die Sprache ist, mit der die Kommunikation zwischen Geräten ermöglicht wird. Sie können in zwei Hauptkategorien unterteilt werden: Netzwerkprotokolle (Wi-Fi, LTE CAT 1, LTE CAT M1, NB-IoT, Bluetooth, ZigBee, LoRaWAN) und Datenprotokolle (AMQP, MQTT, HTTP, CoAP, DDS, LwM2M). Die Datenprotokolle konzentrieren sich hauptsächlich auf den Informationsaustausch, wohingegen die Netzwerkprotokolle die Methoden zur Verbindung von Geräten miteinander oder dem Internet bereitstellen.

2.1.2. Architektur [3]

Die Architektur des IoT variiert je nach Anwendung und Implementierung, weil sie an die spezifischen Anforderungen (Sicherheitsmaßnahmen, Protokolle zur Geräteverwaltung, Edge-Computing-Lösungen) eines Projekts angepasst wird. Man kann sie jedoch trotzdem allgemein in folgende Schichten unterteilen:

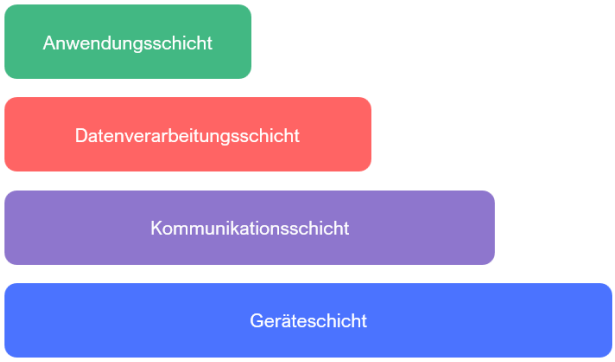


Abbildung 1: IoT-Architektur

1. **Geräteschicht**: Dies ist die unterste Schicht und umfasst die physischen IoT-Geräte, Sensoren und Aktoren, um Daten aus ihrer Umgebung zu sammeln und Aktionen auszuführen. Dabei bildet sie die Schnittstelle zwischen der physischen Welt und dem digitalen IoT-System.
2. **Kommunikationsschicht**: In dieser Schicht werden die Daten, welche von den IoT-Geräten erfasst wurden, übertragen. Dies kann kabelgebunden (Ethernet) oder drahtlos (Wi-Fi, Bluetooth, LoRaWAN, 5G) erfolgen. Die Kommunikationsschicht ermöglicht den Datentransfer von den Geräten zu den darüberliegenden Schichten und umgekehrt.
3. **Datenverarbeitungsschicht**: Hier werden die empfangenen Daten verarbeitet und weitergeleitet. Diese Schicht kann sich in der Cloud oder auf lokalen Servern befinden und beinhaltet die Datenverarbeitung und -speicherung.
4. **Anwendungsschicht**: Die Anwendungsschicht ist die oberste Schicht und stellt die Benutzerschnittstelle dar. Hier werden Anwendungen entwickelt, welche auf den IoT-Daten basieren. Dies können beispielsweise Anwendungen für Smart Homes, Industrieautomatisierung, Gesundheitswesen, Logistik und viele weitere Lebensbereiche sein.

2.2. Embedded System

2.2.1. Arduino Uno [4, 5, 6, 7, 8]

Der Arduino Uno stellt ein vielseitiges Mikrocontroller-Board dar, welches auf dem ATmega328P-Chip basiert und diverse Funktionalitäten bietet. Mit insgesamt 14 digitalen Ein-/Ausgangspins, von denen 6 als PWM-Ausgänge konfigurierbar sind, sowie 6 analogen Eingängen, einem präzisen 16-MHz-Keramikresonator und einer Vielzahl von Schnittstellen, einschließlich USB-Anschluss, Strombuchse, ICSP-Header und Reset-Taste, bietet das Board sämtliche erforderlichen Ressourcen zur umfassenden Unterstützung des Mikrocontrollers. Die Anbindung an einen Computer mittels eines USB-Kabels oder die Stromversorgung über einen AC-DC-Adapter sowie Batterie ermöglichen eine mühelose Inbetriebnahme für Entwicklungsprojekte. Es bietet eine Möglichkeit, Experimente durchzuführen, ohne übermäßige Bedenken hinsichtlich etwaiger Fehlschläge hegen zu müssen. Im äußersten Fall besteht die Option, den ATmega328P-Chip zu einem geringen finanziellen Aufwand zu ersetzen und das Projekt nahtlos fortzusetzen. Die Offenlegung des Projekts gemäß den Open-Source-Prinzipien fördert die weite Verbreitung und unterstützt eine facettenreiche Bandbreite an Projekten innerhalb der Entwicklergemeinschaft [4].

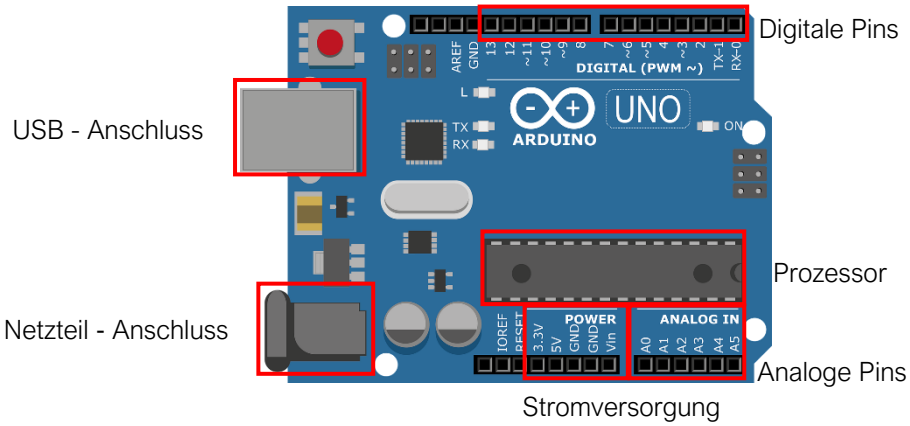


Abbildung 2: Arduino

Tabelle 5: Datenblatt des Arduinos

Mikrocontroller	ATmega328P
Betriebsspannung	5V
Eingangsspannung (empfohlen)	7-12V
Eingangsspannung (maximal)	6-20V
Digital E/A Pins	14 (davon 6 PWM-Ausgang)
PWM Digital E/A Pins	6
Analog E/A Pins	6
DC Strom pro E/A Pin	20 mA
DC Strom für 3.3V Pin	50 mA
Flash Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz
CPU	8-bit
Länge	68,6 mm
Breite	53,4 mm
Gewicht	25 g
Stromverbrauch im Normalbetrieb	47,6 mA

Der ATmega328P-Chip ist mit einem vorprogrammierten Bootloader ausgestattet, welcher das Hochladen neuer Software ermöglicht, ohne dass ein externer Hardware-Programmierer erforderlich ist. Darüber hinaus zeichnet sich der Arduino mittels seiner herausragenden Programmierbarkeit aus, indem er eine Programmiersprache nutzt, welche auf der Basis von C/C++ aufgebaut ist. Dies gewährt Einsteigern sowie erfahrenen Entwicklern eine komfortable Programmierumgebung. Die lebendige Community bietet umfassende Unterstützung in Form von Diskussionsforen, Anleitungen und inspirierenden Projektvorlagen, was die Bewältigung etwaiger Herausforderungen und den

interdisziplinären Wissensaustausch erleichtert. Ein lauffähiges Programm besteht aus verschiedenen Komponenten, welche die Grundstruktur eines jeden Projekts bilden. Sie werden je nach den Anforderungen und den gewünschten Aktionen und Funktionen erweitert und angepasst:

Setup: Die Funktion *setup()* wird einmal beim Start des Programms ausgeführt. Hier werden anfängliche Einstellungen und Konfigurationen vorgenommen, wie das Festlegen von Pins als Ein- oder Ausgänge, die Initialisierung von Sensoren oder Konfiguration von Kommunikationsschnittstellen.

Loop: Die Funktion *loop()* wird kontinuierlich ausgeführt, nachdem *setup()* abgeschlossen ist. Hier wird der Hauptteil des Programms geschrieben und es werden Befehle und Anweisungen platziert, welche in einer Schleife abgearbeitet werden, um das gewünschte Verhalten des Projekts aufrechtzuerhalten.

Variablen: Variablen werden verwendet, um Werte oder Daten im Programm zu speichern. Sie können für verschiedene Zwecke, wie das Speichern von Sensorwerten, Zwischenergebnissen oder Einstellungen genutzt werden.

Kontrollstrukturen: Kontrollstrukturen wie Schleifen *(for, while)* und Verzweigungen *(if, else)* steuern den Programmfluss. Sie ermöglichen es, auf bestimmte Bedingungen zu reagieren und unterschiedliche Aktionen basierend auf diesen Bedingungen auszuführen.

Funktionen: Benutzerdefinierte Funktionen können den Code organisieren und als wiederverwendbare Abschnitte fungieren. Sie werden mit dem Schlüsselwort *void* definiert und können Parameter akzeptieren.

Bibliotheken: Arduino stellt eine Vielzahl von Bibliotheken zur Verfügung, welche spezielle Funktionen und Hardwareunterstützung für Sensoren, Aktoren und sonstige Peripheriegeräte bieten. Sie werden ins Projekt eingebunden, um den Zugriff auf spezifische Funktionen zu erleichtern.

```
1   #include <Arduino.h> // Bibliothek, um Funktionen bereitzustellen
2   int pin = 13; // Variable, um den Pin zu speichern
3
4   void setup() { //Konfigurationen, welche einmal beim Start des
5                  Programms ausgeführt werden
6     pinMode(pin, OUTPUT); //Pin als Ausgang festlegen
7   }
8
9   void loop() { //Hauptcode, welcher kontinuierlich ausgeführt wird
10    digitalWrite(pin, HIGH); //Pin einschalten
11    delay(1000); //Wartezeit
12  }
```

Abbildung 3: Grundgerüst eines Programms in Arduino

2.2.2. **DHT11** [9]

Der DHT11 repräsentiert eine ökonomische Ausführung eines Temperatur- und Feuchtigkeitssensors, welcher gleichwohl präzise Messwerte generiert und vorkalibriert angeboten wird. Infolgedessen wird er in diversen Anwendungsprojekten in Verbindung mit Mikrocontrollern zur Verwendung genommen. Hinsichtlich der Temperaturmessung setzt der Sensor auf einen Heißleiter, den sogenannten Negative Temperature Coefficient Thermistor (NTC). Bei diesem handelt es sich um einen Widerstand, dessen Wert in Abhängigkeit von der Temperatur variiert. Mit steigender Temperatur nimmt die Leitfähigkeit zu, was wiederum zu einer Verringerung des Widerstandswerts führt. Die Berechnung der Temperatur erfolgt durch Umstellung der Formel für den Widerstand. Diese Aufgabe wird unmittelbar von der integrierten Schaltung (IC) innerhalb des Sensors übernommen und die resultierenden Daten werden über einen DATA-Pin an den Mikrocontroller übertragen. Dies geschieht durch eine sequenzielle Übermittlung von Einzelbits mittels HIGH- und LOW-Signalen. Für den Sensor stellt Arduino eine vorgefertigte Bibliothek zur Verfügung, welche es vereinfacht, den DHT11 anzusteuern und Daten auszulesen.

Tabelle 6: Datenblatt des DHT11

Parameter	**Werte**
Spannungsbereich	3 – 5V DC
Feuchtigkeitsmessbereich	20 – 90%
Temperaturmessbereich	0°C – 50°C
Messgenauigkeit Feuchtigkeit	± 5,0%
Messgenauigkeit Temperatur	± 2,0°C

2.2.3. **BH1750** [10]

Der BH1750 stellt ein digitales Instrument dar, konzipiert zur exakten Messung der Helligkeit in Lux, wodurch er sich als äußerst präzise und empfindliche Einrichtung zur Beleuchtungsmessung auszeichnet. Dieser Sensor basiert auf einem integrierten Digital-Analog-Konverter (ADC), welcher die empfangenen Lichtsignale von Fotosensoren in präzise digitale Werte umsetzt. Mit einer bemerkenswerten Auflösung von einem Lux besitzt der BH1750 die Fähigkeit, minimale Helligkeitsveränderungen erfassen. Die Handhabung des Sensors erweist sich als unkompliziert, und er kann mithilfe der I2C-Schnittstelle (Inter-Integrated Circuit) nahtlos mit einem Arduino kommunizieren. In Bezug auf die Spannungsversorgung bietet er eine breite Betriebsmöglichkeit im Bereich von 2,4V bis 3,6V. Im aktiven Betriebsmodus verbraucht der Sensor lediglich 1,2 mA, was ihn auch für batteriebetriebene Anwendungen als adäquate Wahl erscheinen lässt. Der Sensor erweist sich in vielerlei Hinsicht als nützlich und findet Verwendung in diversen Anwendungsszenarien, wie etwa Wetterstationen, welche das Umgebungslicht akkurat vermessen oder in Beleuchtungssystemen zur Anpassung der Helligkeit in Innenräumen.

2.2.4. **ESP8266** [11, 12]

Der ESP8266 ist nicht nur ein weitverbreiteter Mikrocontroller mit einer integrierten Wi-Fi-Schnittstelle, sondern überzeugt auch durch seine beachtliche Leistungsfähigkeit. Er ist mit einem L106 32-Bit RISC-Mikroprozessor und einem Wi-Fi-Sendeempfänger ausgestattet und verfügt darüber hinaus über einen vollständigen TCP/IP-Stack und integrierte TCP/UDP-Serverfunktionen. Diese Eigenschaften ermöglichen eine unkomplizierte Realisierung von Projekten im Bereich des Internet of Things (IoT) und bieten vielfältige Anwendungsmöglichkeiten. Vorgefertigte Firmware-Images wie Tasmota, ESPEasy oder WLED wird die Integration in Projekte erheblich vereinfacht. Die Boards bieten ein breites Spektrum an hardwareseitigen Schnittstellen wie UART, I2C und SPI, welche die Kommunikation mit externen Geräten und die Anbindung von Sensoren ermöglichen. Darüber hinaus sind digitale Ein- und Ausgangspins sowie ein analoger Pin vorhanden. Demnach eignet sich der ESP8266 perfekt um Sensordaten senden, online sammeln und auswerten zu können.

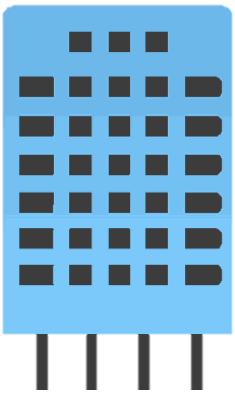


Abbildung 6: DHT11 [9]

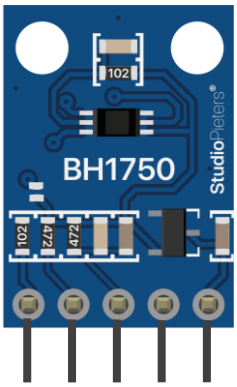


Abbildung 5: BH1750 [10]

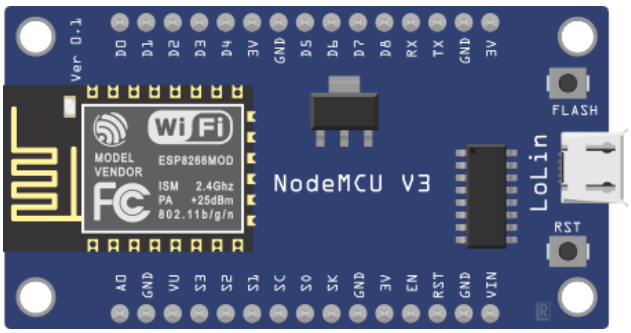


Abbildung 4: ESP8266 [13]

2.3. Datenkommunikation mittels HTTP [14, 15]

HTTP (Hypertext Transfer Protocol) spielt eine zentrale Rolle im World Wide Web, weil es vorrangig dazu verwendet wird, Hypertext-Dokumente, und somit ebenfalls Webseiten, aus dem Internet in einen Webbrowser zu laden. Dies ermöglicht es dem Client, etwaige Anfragen an einen Server zu senden, um eine breite Sammlung von Informationen, wie Texte, Bilder und Videos abzurufen. Dieser kontinuierliche Informationsaustausch bildet das Grundgerüst des modernen Internets, wodurch Benutzer nahtlos auf Informationen im Web zugreifen können.

2.3.1. Methoden

HTTP-Anfragen sind Nachrichten, welche vom Client an den Server gesendet werden. Diese Anfragen enthalten Informationen wie die gewünschte Aktion und die URL der Ressource, auf die zugegriffen werden soll. HTTP-Anfragen verwenden verschiedene Methoden:

Tabelle 7: HTTP-Methoden

Methode	Beschreibung
GET	Daten abrufen
POST	Daten senden oder Ressource erstellen
PUT	Ressource aktualisieren oder erstellen
DELETE	Ressource löschen
PATCH	Teil der Ressource aktualisieren
HEAD	Header-Informationen abrufen
OPTIONS	Unterstützte Methoden abrufen
CONNECT	Netzwerkverbindung herstellen (meistens für Proxys)
TRACE	Anfragen verfolgen (Debugging)

2.3.2. Statuscodes

HTTP-Antworten sind Nachrichten, welche von einem Server an einen Client zurückgesendet werden, um auf eine Anfrage zu reagieren, welche der Client zuvor initiiert hat. Eine zentrale Rolle bei diesen Antworten spielen sogenannte Statuscodes. Sie können als Mitteilungen des Servers an den Client betrachtet werden, um mitzuteilen, wie die Anfrage verarbeitet wurde. Die Fehlererkennung und -behandlung sind entscheidend, um eine robuste Kommunikation sicherzustellen. Statuscodes werden verwendet, um den Status von Anfragen und Antworten anzuzeigen und auftretende Fehler dementsprechend zu handeln.

Tabelle 8: HTTP-Statuscodes

Methode	Beschreibung
1xx (Informational) z.B. 100 (Continue)	Vorläufige Informationen, dass die Anfrage empfangen wurde, jedoch weitere Aktionen erforderlich sind
2xx (Success) z.B. 200 (OK)	Anfrage wurde erfolgreich bearbeitet
3xx (Redirection) z.B. 302 (Found)	Deuten auf Umleitungen hin und erfordern zusätzliche Aktionen des Clients
4xx (Client Error) z.B. 400 (Bad Request)	Fehler in der Anfrage des Clients
5xx (Server Error) z.B. 500 (Internal Server Error)	Serverfehler, bei denen die Anfrage an sich korrekt war, jedoch der Server trotzdem Probleme bei der Verarbeitung hatte,

JSON (JavaScript Object Notation) ist ein weit verbreitetes Datenformat, das sich für strukturierte Daten eignet, welche in Objekten verpackt und zwischen Client und Server übertragen werden.

2.4. Webanwendung

2.4.1. Frontend

2.4.1.1. **Vue** [16, 17, 18, 19, 20, 21, 22]

Vue ist ein JavaScript-Framework, das auf faszinierende Technologien zurückgreift. Im kommenden Kapitel werden die vier wichtigsten dieser Technologien näher erläutert. Hierzu zählt eine komponentenbasierte Architektur, um die Benutzerschnittstelle in wiederverwendbare Teile zu gliedern und isoliert voneinander entwickelt, getestet und gewartet werden, was den Entwicklungsprozess vereinfacht. Dies ähnelt der Verschachtelung von HTML-Elementen, jedoch implementiert Vue sein eigenes Modell, um Benutzerschnittstelle und Logik in jeder einzelnen Komponente zu kapseln.

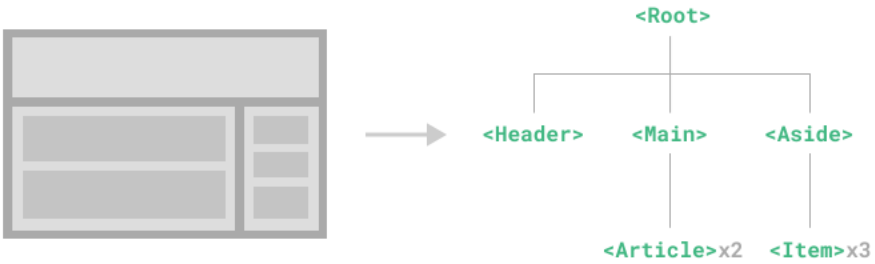


Abbildung 7: Komponentenbaum

Die reaktive zweiseitige Datenbindung (two-way binding) tritt auf, wenn Daten nicht nur vom JavaScript zum DOM (Document Object Model) gebunden werden, sondern auch in umgekehrter Richtung. Im folgenden Codebeispiel stellt die Direktive *v-model* sicher, dass Änderungen in der Benutzeroberfläche reaktiv angezeigt werden. Bei einer Änderung des Werts in der Benutzeroberfläche erfolgt eine automatische Aktualisierung in der zugrunde liegenden Logik, und umgekehrt, bei einer Wertänderung in der Logik zeigt sich die Aktualisierung im Benutzerschnittstelle. Es gibt eine Vielzahl an Direktiven, inklusive der Möglichkeit, eigene Direktive zu erstellen.

```
1  <div v-if="message">{{ message }}<div>
2  <div v-model="message">{{ message }}<<div>
3  <div v-for="message in messages">{{ message }}<div>
```

Abbildung 8 : Datenbindung mittels Direktiven

SFC (Single-File Component oder *.vue-Datei) ist ein spezielles Dateiformat, welches HTML-Template, JavaScript-Logik und CSS-Styling einer Komponente in einer einzigen Datei bündelt. Dies führt zur Zusammenführung von natürlicherweise zusammengehörenden Aspekten, weil sich in der modernen Webentwicklung aufgezeigt hat, dass es anstelle einer Aufteilung des Codes in drei große, miteinander verflochtene Schichten sinnvoller ist, sie in lose gekoppelte Komponenten aufzuteilen und diese miteinander zu kombinieren.

```
1   <script>
2   import { ref } from 'vue'
3   const message = ref('Hello World!')
4   </script>
5
6   <template>
7     <div class="message">{{ message }}</p>
8   </template>
9
10  <style>
11  .message {color: red}
12  </style>
```

Abbildung 9: Message.vue

Wenn der Benutzer mit einer Seite interagiert, wechseln Elemente ihren Zustand. Der Browser übernimmt die Verantwortung, folglich erzeugte Informationsänderungen zu aktualisieren und am Bildschirm anzuzeigen. Eine Herausforderung besteht jedoch darin, dass die Aktualisierung des gesamten DOM eine zeitaufwändige Aufgabe sein kann. Vue hat infolgedessen eine virtuelle Kopie des ursprünglichen DOM der Webseite, wodurch eine angemessene Anzahl von Änderungen vorgenommen werden kann, ohne den gesamten DOM neu zu rendern. Dies resultiert in besserer Leistung sowie Geschwindigkeit und schnelleren DOM-Operationen.

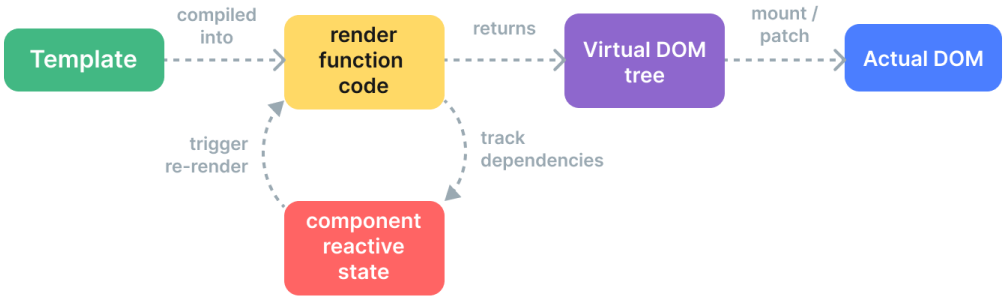


Abbildung 10: Render Pipeline

2.4.1.2. Nuxt [23]

Nuxt ist ein serverseitiges Rendering-Framework, welches auf Vue aufsetzt. Es vereinfacht die Verwaltung komplexer Konfigurationen im Zusammenhang mit asynchronen Daten, Middleware und Routing. Darüber hinaus erleichtert es die Strukturierung von Webanwendungen:

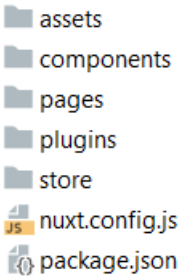


Abbildung 11: Verzeichnisstruktur

In den Assets werden nicht kompilierte Ressourcen wie Bilder, Schriftarten und CSS-Dateien verwaltet. Vue-Komponenten werden in den Components abgelegt und können in Seiten, Layouts und anderen Teilen der Anwendung wiederverwendet werden. Layouts ermöglicht die Trennung unterschiedlicher Anwendungsstrukturen, insbesondere für authentifizierte und nicht authentifizierte Benutzer, und Pages bildet die Grundlage des Routing-Systems und generiert Routen automatisch aus vue-Dateien, weswegen eine Datei in dem Verzeichnis ebenfalls einer Seite in der Anwendung resultiert.

Plugins kann vor der Instanziierung der Anwendung eingefügt werden, beispielsweise um die App zu erstellen und alle globalen Bibliotheken zu importieren und ihr zuzuordnen. Der Store speichert den Anwendungszustand und fungiert oft als Schnittstelle zwischen Front- und Backend.

2.4.1.3. Tailwind CSS

Tailwind CSS ist ein CSS-Framework, welches im Gegensatz zu Bootstrap oder anderen ähnlichen Frameworks nicht nur vorgefertigte Komponenten bereitstellt, sondern man sie ebenfalls individuell und an den eigenen Vorstellungen anpassen und inline stylen kann. Im nachfolgenden Codebeispiel ist die klare Lesbarkeit erkennbar, und daneben ist das resultierende Ergebnis.

```
1  <div class="bg-[#BBF7D0]
2              border-[1px] border-[#000000] rounded-[6px]
3              w-[16px] h-[16px]"/>
```

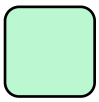


Abbildung 12: Tailwind CSS inline styling

Darüber hinaus ist Flex in Kombination mit responsivem Design neben der einfachen Gestaltung eine der wichtigsten Technologien, weil sie das Layout intuitiv vereinfacht und aufwendiges, manuelles Styling mit CSS ersetzt. Im nachfolgenden Codebeispiel werden alle Inhalte am Desktop in einer Reihe und auf mobilen Geräten in Spalten angezeigt. Hierbei steht *lg* für den Umbruch, der in diesem Fall bei 1024px erfolgt.

```
1  <div class="flex flex-col lg:flex-row"/>
```

Abbildung 13: Flex im responsiven Design

2.4.2. Backend

2.4.2.1. Datenbankmanagement mit MySQL [24, 25]

MySQL ist ein weit verbreitetes relationales Datenbankmanagementsystem (RDBMS). Die Datenbank ist so konzipiert, dass sie verschiedene Tabellen zur Speicherung unterschiedlicher Daten enthält. Die Definition der Tabellenspalten und Datentypen erfolgt entsprechend den Anforderungen. Um die Datenkonsistenz und -integrität sicherzustellen, wird die Datenbank normalisiert. Dies bedeutet, dass Redundanz in den Daten minimiert wird, und Beziehungen zwischen Tabellen werden sorgfältig geplant. SQL wird verwendet, um Daten aus der MySQL-Datenbank abzurufen, einzufügen, zu aktualisieren und zu löschen. Dies ermöglicht es dem Backend, auf Benutzeranfragen zu reagieren und die entsprechenden Daten bereitzustellen.

2.4.2.2. Sicherheits- und Datenschutzmechanismen [26]

Eine entscheidende Komponente ist die umfassende Kontrolle des Datenbankzugriffs über geschützte Kommunikationskanäle. Dies stellt sicher, dass Nutzer lediglich auf jene Daten zugreifen können, für welche sie explizit autorisiert sind. Die Verwaltung und Überwachung von Zugriffsrechten erfolgen in einem akribischen Prozess. Empfindliche Informationen wie Passwörter und personenbezogene Daten unterliegen einer kryptografischen Verschlüsselung vor ihrer Speicherung in der Datenbank. Diese Maßnahme gewährleistet, dass selbst im Falle eines Datenbanklecks die Integrität und

Vertraulichkeit der Informationen gewahrt bleiben. Die Sicherheit des Datenbankmanagementsystems, insbesondere MySQL, wird durch konstante Aktualisierungen und Patches gewährleistet. Diese Maßnahme zielt darauf ab, potenzielle Sicherheitslücken zu schließen und präventive Abwehrmechanismen gegen mögliche Angriffe zu implementieren. Eine aktuelle Datenbanksoftware stellt somit ein kritisches Element für die Sicherheit des Gesamtsystems dar.

2.4.2.3. Datenverarbeitung und -validierung [27, 28]

Das Backend verarbeitet und validiert Daten, bevor sie in die Datenbank eingefügt werden. Dies ist wichtig, um die Datenkonsistenz und -qualität sicherzustellen. Ein effizientes Datenbankmanagement in Kombination mit Sicherheitsmaßnahmen gewährleistet eine zuverlässige und sichere Verarbeitung der Daten. MySQL als Datenbanksystem bietet eine robuste Grundlage für die Speicherung und Verwaltung von Daten. Beispiele für Datenverarbeitung und Validierung im Backend sind:

Formatvalidierung: Bevor Daten in die Datenbank geschrieben werden, werden sie auf das richtige Format und die Einhaltung der Datentypen überprüft. Dies verhindert Fehler und Inkonsistenzen.

Berechnungen und Aggregationen: Das Backend kann auch Berechnungen durchführen, um beispielsweise Durchschnittswerte von Umweltbedingungen zu ermitteln oder Statistiken über das Pflanzenwachstum zu generieren.

Eingabevalidierung: Bei Benutzereingaben werden verschiedene Sicherheitsprüfungen durchgeführt, um Sicherheitslücken wie SQL-Injection oder Cross-Site Scripting (XSS) zu verhindern.

3. Praktische Ausarbeitung

3.1. Meilenstein 1

3.1.1. Temperatur und Luftfeuchtigkeit

Es wurde zuerst die physische Verbindung des DHT11 mit dem Arduino durchgeführt, um folglich das zugehörige Programm mit der Einbindung der Bibliothek *DHT sensor library by Adafruit* zu entwickeln. Dies ermöglichte eine einfachere Interaktion mit dem Sensor und trug dazu bei, die Genauigkeit der Messungen zu erhöhen.

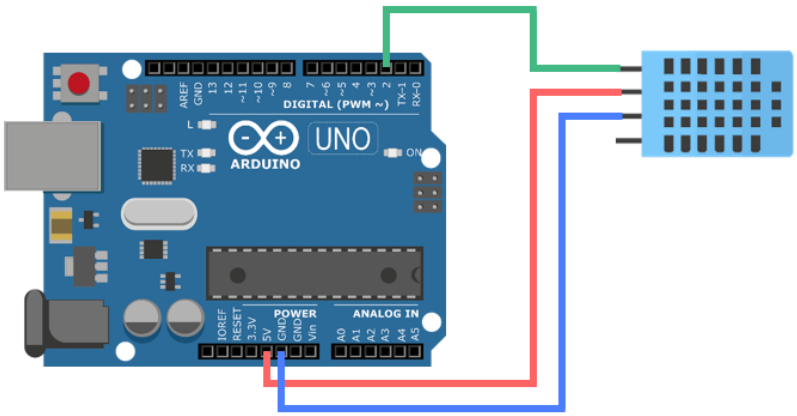


Abbildung 14: Verkabelung des DHT11

```
#include <DHT.h>

#define DHT_PIN 2 //Pin des DHT11
#define DHT_TYPE DHT11 //DHT11
DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(9600); //Start der seriellen Kommunikation
  dht.begin(); //Initisialisierung des DHT11
}

void loop() {
  //Messungen einlesen und in einer Variable speichern
  float luftfeuchtigkeit = dht.readHumidity();
  float temperatur = dht.readTemperature();

  //Messungen in der Konsole ausgeben
  Serial.println("Luftfeuchtigkeit: " +
      String(luftfeuchtigkeit) + " %");
  Serial.println("Temperatur: " +
      String(temperatur) + " °C");

  delay(4000); //Wartezeit von 4 Sekunden
}
```

Abbildung 15: Programm des DHT11

3.1.2. Projekteinrichtung des Frontends

Bevor mit der Projekteinrichtung begonnen werden kann, muss bereits Node installiert sein. Node fungiert als Laufzeitumgebung für JavaScript und kann unkompliziert von der offiziellen Webseite bezogen sowie eingerichtet werden. Danach kann man bereits im Terminal zum gewünschten Verzeichnis navigieren und den Befehl mit dem Projektnamen *web-application* ausführen.

```
> npm nuxi init web-application
```

Abbildung 16: Projektinitialisierung

Nachfolgend kann das initialisierte Projekt in Visual Studio Code geöffnet werden, um mit der Entwicklung zu beginnen und die Webseite mit folgendem Befehl anzuzeigen.

```
> npm run dev
```

Abbildung 17: Projektstart

Demzufolge werden die wichtigsten Bibliotheken installiert und in *nuxt.config.ts* integriert, um Komponenten, Icons und Styling bereitzustellen.

```
> npm install primevue
> npm install primeicons
> npm install @nuxtjs/tailwindcss
```

```
export default defineNuxtConfig({
  modules: ['@nuxtjs/tailwindcss'],
  build:{transpile: ['primevue']},
  css:[
    'primevue/resources/themes/saga-blue/theme.css',
    'primevue/resources/primevue.min.css',
    'primeicons/primeicons.css'
  ]
})
```

Abbildung 18: nuxt.conig.ts

In *plugins/index.js* werden PrimeVue und später sonstige Komponenten (aus testzwecken Message) importiert und der *App* zugeordnet.

```
import {defineNuxtPlugin} from 'nuxt/app'
import PrimeVue from 'primevue/config'
import Message from 'primevue/message'

export default defineNuxtPlugin((nuxtApp) => {
    nuxtApp.vueApp.use(PrimeVue).component('Message', Message)
})
```

Abbildung 19: plugin/index.js

Letztlich kann man unter *pages/index.vue* die Funktionalität von Vue, Nuxt, Tailwind CSS und den Komponenten sowie Icons von PrimeVue mit folgendem Code testen und das Resultat auf der Webseite betrachten. Demnach ist das Projekt eingerichtet und es kann mit der Implementierung des Frontends begonnen werden.

```
<template>
    <Message>
        <div class="text-[#044868]">Hello World!
        <i class="pi pi-globe"/></div>
    </Message>
</template>
```

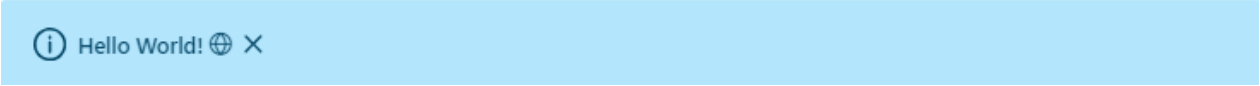


Abbildung 20: Anzeige des eingerichteten Projekts

3.2. Meilenstein 2

3.2.1. Startseite

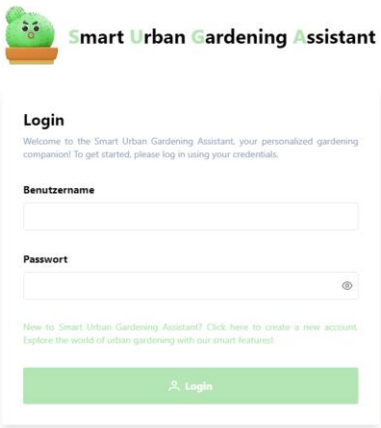


Abbildung 21: Login am Desktop

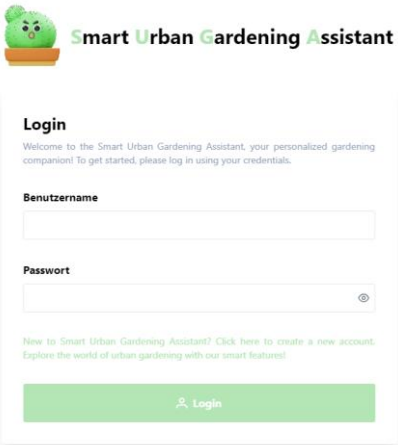


Abbildung 23: Anmeldung am Smartphone

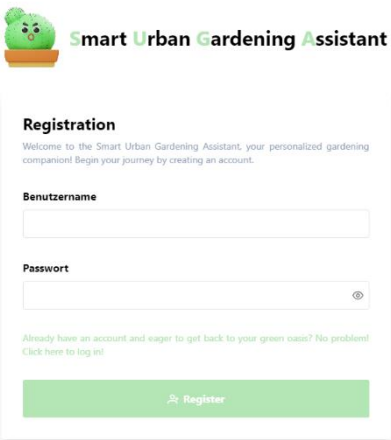


Abbildung 22: Registrierung am Smartphone

Anfangs wurde eine repräsentative Farbe *plantGreen* für die Anwendung ausgewählt und global in *tailwind.config.ts* festgelegt. Durch eine einzige Anpassung des Farbcodes können sämtliche Farben der Anwendung entsprechend modifiziert werden. Der folgende Befehl initialisiert die besagte Datei, wobei ebenfalls Abstände, Schriftarten, Eckenradien und weiteren gestalterischen Aspekte zentral festgelegt werden, um die Übersichtlichkeit und Konsistenz der Designelemente zu gewährleisten.

```
> npx tailwindcss init
```

```
export default {
  theme: {
    extend: {
      colors: {
        plantGreen: '#B4E5B4', //primary color
      },
    },
  },
}
```

Abbildung 24: tailwind.config.ts

Die Benutzeroberfläche soll teilweise auf Karten basieren. Dafür wurde eine entsprechende Komponente *Card.vue* entwickelt, welche beispielsweise beim Login den Anmelde- oder Registrierungsbereich umfassen. Hierbei besteht die Möglichkeit, optional anzugeben, ob die Karte einen Schatten oder Hover-Effekt aufweisen soll. Letzteres hebt den Rand farblich hervor, um zu signalisieren, dass die Karte anklickbar ist. Grundsätzlich sind beide Funktionen initial deaktiviert.

```
<template>
  <div :class="{'shadow-lg': props.shadow,
                'cursor-pointer border hover:border
                 hover:border-plantGreen': props.hover }">
    <slot/>
  </div>
</template>

<script setup>
const props = defineProps({
  shadow: {
    type: Boolean,
    required: false,
    default: false,
  },
  hover: {
    type: Boolean,
    required: false,
    default: false,
  }
})
</script>
```

Abbildung 25: Card.vue

3.2.1.1. Login (Component)

Login.vue beginnt mit einer Überschrift und einer kurzen Erläuterung. Der hart codierte Text muss zu einem späteren Zeitpunkt mittels Schlüssel ersetzt werden, weil dies nicht nur empfehlenswert, sondern unerlässlich ist, um die Implementierung mehrsprachiger Unterstützung zu ermöglichen.

```
<template>
  <Card shadow class="rounded p-8">
    <div class="flex-col space-y-8">
      <div class="space-y-2">
        <div class="text-2xl font-bold">Login</div>
```

```
        <div class="text-sm text-slate-400 text-justify">
          Welcome to the Smart Urban Gardening Assistant,
          your personalized gardening companion!
          To get started, please log in using your credentials.
        </div>
      </div>
```

Abbildung 26: Beschreibung der Startseite

Daraufhin erfolgt die Eingabe des Benutzernamens und Passworts, welche mittels *InputText* und *Password* realisiert werden. Dabei kann für das Passwort zwischen Klartext- und verschlüsselter Anzeige gewechselt werden.

```
      <div class="flex flex-col space-y-2">
        <label for="username" class="font-bold">Benutzername</label>
        <InputText id="username" v-model="username"
                   class="border py-2 px-4 rounded"/>
      </div>
      <div class="flex flex-col space-y-2">
        <label for="password" class="font-bold">Passwort</label>
        <Password id="password" v-model="password" toggleMask
                  :feedback="false" class="border rounded"/>
      </div>
      <div class="text-sm text-justify cursor-pointer text-plantGreen"
           @click="toggleLogin"> New to Smart Urban Gardening Assistant?
           Click here to create a new account. Explore the world of urban
           gardening with our smart features!
      </div>
```

Abbildung 27: Benutzername und Passwort der Startseite

Es folgt ein anklickbarer Text, bei dem *toggleLogin* aufgerufen wird, um zur Registrierung zu wechseln, falls noch kein Account existiert. Darunter befindet sich der *Button* zur Anmeldung, der *login* aufruft. Letztlich erfolgt die Anzeige einer *Message* nur im Falle fehlerhafter Eingabedaten *wrongCredentials*.

```
      <div class="text-sm text-justify cursor-pointer text-plantGreen"
           @click="toggleLogin"> New to Smart Urban Gardening Assistant?
           Click here to create a new account. Explore the world of urban
           gardening with our smart features!
      </div>
      <Button @click="login"
        class="w-full bg-plantGreen text-center rouneded p-4">
        <div class="flex items-center space-x-2 mx-auto">
          <i class="pi pi-user text-white"/>
          <div class="text-white text-l font-bold">Login</div>
        </div>
      </Button>
      <Message v-if="wrongCredentials" severity="error" :closable="false">
        <div class="text-sm">Oops!
        It seems that the username or password is incorrect </div>
      </Message>
    </div>
  </Card>
</template>
```

Abbildung 28: Link, Button und Meldung der Startseite

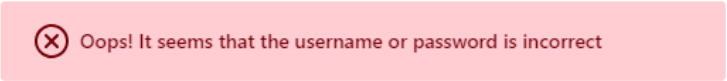


Abbildung 29: Meldung bei fehlerhaften Eingabedaten der Startseite

Die Benutzer *users* werden in einer Liste hinterlegt, welche später vom Backend bezogen wird. Darüber hinaus müssen Benutzername *username* und Passwort *password* mit den Eingabefeldern verknüpft werden.

```
<script setup>
import Card from "../components/Card.vue"
import { ref } from "vue"
import { useRouter } from "vue-router"

const users = ref([])
const username = ref()
const password = ref()
const wrongCredentials = ref(false)
const router = useRouter()
```

Abbildung 30: Variablen des Logins

Toggle-login sendet mittels *emit* ein Signal an die übergeordnete Komponente. Infolge der Anmeldung *login* wird der Benutzer anhand des Benutzernamens gesucht. Falls der Benutzer nicht *null* ist und das Passwort übereinstimmt, erfolgt mittels *router* die Weiterleitung zur Hauptseite, indem der Name der Page an die URL angehängt wird. Andernfalls wird ein boolescher Wert *wrongCredentials* auf *true* gesetzt, um eine entsprechende Meldung anzuzeigen.

```
const emit = defineEmits(["toggle-login"])
const toggleLogin = () => emit("toggle-login", true)
function login(){
  const user = users.value.find(user => user.username === username.value)
  if (user && user.password === password.value) router.push('/main')
  else wrongCredentials.value = true
}
</script>
```

Abbildung 31: Funktionen des Logins

Der Style von Password erfolgt indem die ursprüngliche Klasse der Komponente mit dem Zusatz *!important* überschrieben wird.

```
<style>
.p-password .p-password-input{
  width: 100% !important;
  padding: 8px 16px !important;
}
</style>
```

Abbildung 32: Style des Logins

3.2.1.2. Registration (Component)

Die Struktur von *Registration.vue* ähnelt der von *Login.vue*, weswegen nur die wesentlichen Unterschiede dokumentiert werden. Bei der Festlegung des neuen Passworts wurde eine Empfehlung hinzugefügt, um ein sicheres Passwort zu wählen.

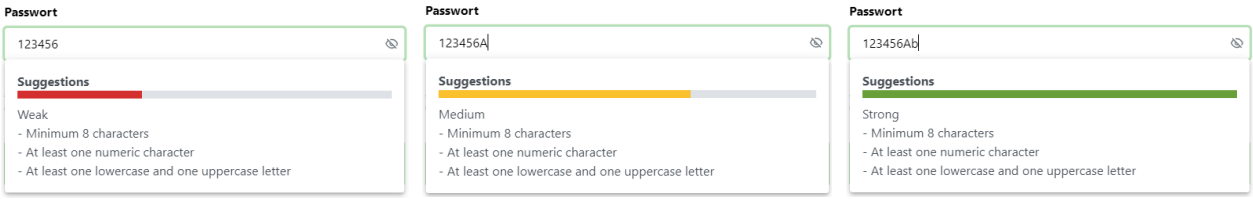


Abbildung 33: Empfehlungen eines sicheren Passworts

```
<Password id="password" v-model="password" toggleMask class="border rounded">
   <template #header>
      <div class="font-bold">Suggestions</div>
   </template>
   <template #footer>
      <ul>
         <li>- Minimum 8 characters</li>
         <li>- At least one numeric character</li>
         <li>- At least one lowercase and one uppercase letter</li>
      </ul>
    </template>
</Password>
```

Abbildung 34: Erweiterung von Password zur Registrierung

Infolge der Registrierung *register* wird der Benutzer anhand des Benutzernamens gesucht. Falls der Benutzername noch verfügbar ist, weil kein Benutzer mit dem angegebenen Benutzernamen gefunden und beide Felder ausgefüllt wurden, soll der neue Benutzer hinzugefügt und zur Hauptseite weitergeleitet werden.

```
function register(){
  if (username.value !== null && password.value !== null &&
      !users.value.some(user => user.username === username.value)){
        users.value.push({ username: username.value,
                           password: password.value })
        router.push('/main') //zur Hauptseite weiterleiten
      }
  else wrongCredentials.value = true //Benutzername bereits vergeben
}
```

Abbildung 35: Funktion zur Registerierung

3.2.1.3. Startseite (Page)

In der Page werden beide Komponenten miteinander verbunden. In der Desktopansicht wird auf der linken Hälfte das Bild, auf der rechten Hälfte das Icon mit Überschrift und je nach booleschem Wert von *showRegistration* die entsprechende Komponente angezeigt. In der Smartphoneansicht wird das Bild bewusst entfernt, um Platz zu sparen.

```
<template>
  <div class="flex items-center justify-center min-h-screen">
    <img src="@/assets/images/plantsBalcony.jpg"
         class="min-h-screen w-1/2 object-cover hidden lg:block">
    <div class="flex flex-col items-center space-y-8 lg:w-1/2">
      <div class="flex items-center text-3xl font-bold text-center">
        <img src="@/assets/images/plantIcon.png"
             class="w-[80px] h-[95px] object-cover me-4">
        <div class="flex space-x-2">
          <span class="text-plantGreen">S</span>mart
          <span class="text-plantGreen">U</span>rban
          <span class="text-plantGreen">G</span>ardening
          <span class="text-plantGreen">A</span>ssistant
        </div>
      </div>
      <div class="w-full max-w-xl mx-auto">
        <Login v-if="!showRegistration" @toggle-login="toggleLogin"/>
        <Registration v-else @toggle-login="toggleLogin"/>
      </div>
    </div>
  </div>
</template>
```

Abbildung 36: Template der Startseite

Durch die *emits toggle-login* der Komponenten kann der Wert von showRegistration überschrieben werden, um sicherzustellen, dass jederzeit die richtige Komponente angezeigt wird. Dies ermöglicht eine zentrale Verwaltung der Anzeige.

```
<script setup>
import { ref } from 'vue'

const showRegistration = ref(false)
const toggleLogin = (bool) => showRegistration.value = bool
</script>
```

Abbildung 37: Skript der Startseite

3.2.2. Hauptseite

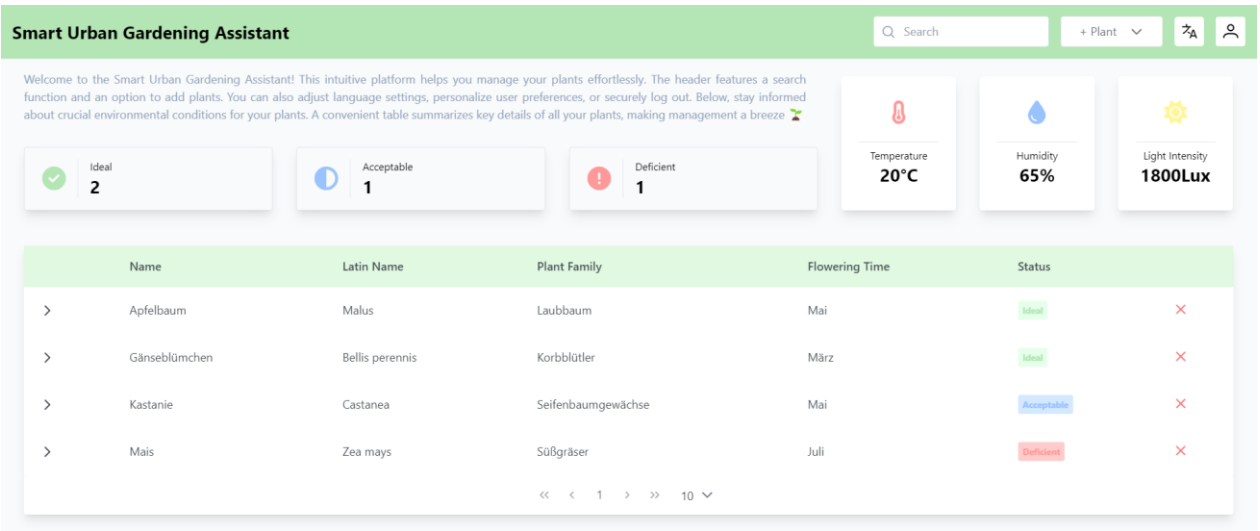


Abbildung 38: Hauptseite

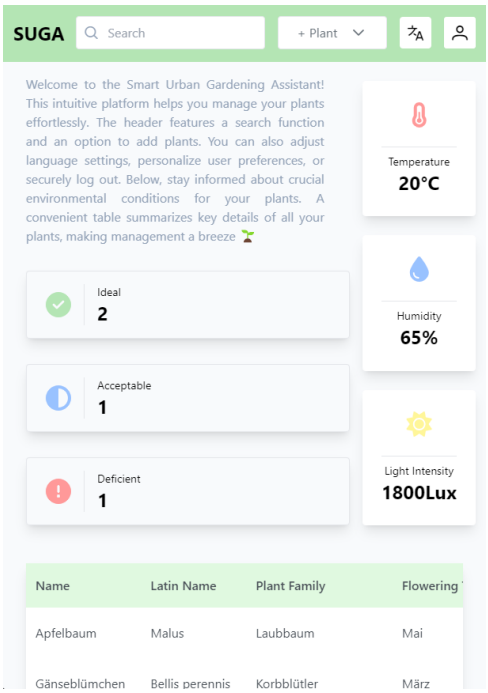


Abbildung 40: Smartphoneansicht der Hauptseite

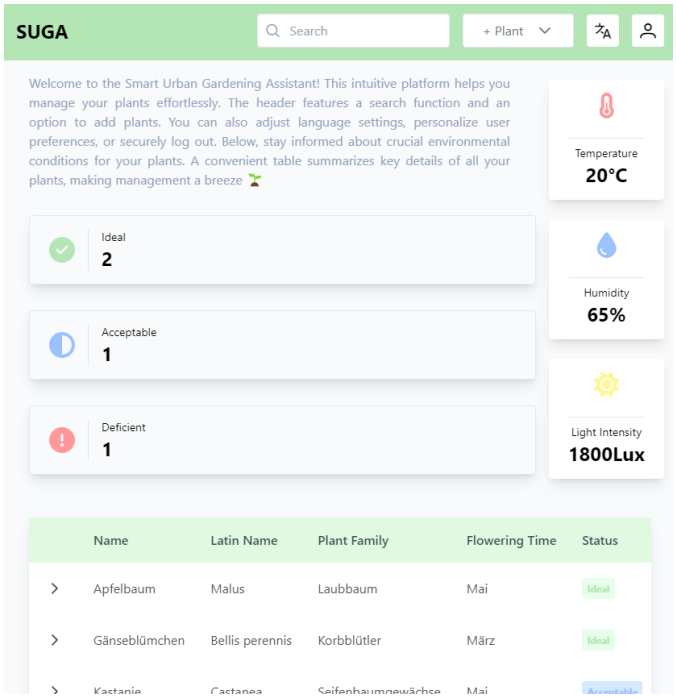


Abbildung 39: Tabletansicht der Hauptseite

3.2.2.1. Header (Component)

Header.vue initiiert auf der linken Hälfte in der Desktopansicht dem vollen Titel und präsentiert auf dem Smartphone eine prägnante Kurzform der Initialen desselben.

```
<template>
  <div class="flex justify-between items-center bg-plantGreen p-4">
    <div class="text-2xl font-bold lg:hidden">SUGA</div>
    <div class="text-2xl font-bold hidden lg:block">
          Smart Urban Gardening Assistant
    </div>
```

Abbildung 41: Titel der Hauptseite

Danach ergibt sich die Möglichkeit, die Pflanzentabelle mittels einer Sucheingabe zu filtern. Diese wurde durch Verwendung eines *InputText* umgesetzt, bei der die Eingabe in *searchInput* gespeichert liegt. Darüber hinaus ist das Eingabefeld mit einem unverkennbaren *Icon* versehen.

```
7   <div class="flex space-x-4 items-center">
8         <div class="relative">
9           <i class="absolute pi pi-search left-3 top-1/2 transform
10                    -translate-y-1/2 text-slate-400"/>
11          <InputText v-model="searchInput" placeholder="Search"
12                     class="border ps-10 py-2 px-4 rounded"/>
13        </div>
```

Abbildung 42: Pflanzen suchen in der Hauptseite

Im *Multiselect* werden sämtliche verfügbaren Pflanzen nach Namen alphabetisch sortiert angezeigt. Mithilfe der Suchfunktion ist es möglich, eine oder mehrere Pflanzen zu suchen und auszuwählen. Die Auswahl kann mittels Hinzufügens oder Entfernen gesteuert und in *plantChosen* verwaltet werden, jedoch werden die Änderungen erst nach Betätigen des Buttons *addPlant* in die Pflanzentabelle übernommen. Die ersten drei ausgewählten Pflanzen erscheinen in einer Vorschau, in der sie ebenso entfernt werden können.

Abbildung 43: Pflanze hinzufügen in der Hauptseite

```
8         <MultiSelect v-model="plantChosen" :options="plantOptions"
9                 placeholder="+ Plant" optionLabel="name" display="chip"
10                class="border px-4 rounded" :filter="true" :showToggleAll="false"
11                :maxSelectedLabels="3">
12          <template #footer>
13              <Button type="button" @click="addPlant"
14                      class="w-full bg-plantGreen p-2">
15                <div class="flex items-center space-x-2 mx-auto">
16                  <i class="pi pi-plus text-white"></i>
17                   <div class="text-white font-bold text-l">Plant</div>
18                </div>
19              </Button>
20          </template>
21        </MultiSelect>
```

Abbildung 44: Pflanze hinzufügen in der Hauptseite

Im *Header* befinden sich zuletzt zwei Quadrate mit *Icons*, welche die Optionen für Sprachauswahl und Benutzerprofil repräsentieren. Durch einen Klick wird entweder *showLanguagePanel* oder *showSettingsPanel* ausgeführt.

```
22      <div class="w-[40px] h-[40px] bg-white flex rounded
23                  items-center justify-center">
24        <i class="pi pi-language cursor-pointer text-[20px]"
25           @click="showLanguagePanel" />
26      </div>
27      <div class="w-[40px] h-[40px] bg-white flex rounded
28                  items-center justify-center">
29        <i class="pi pi-user cursor-pointer text-[20px]"
30           @click="showSettingsPanel"/>
31      </div>
32    </div>
33  </div>
```

Abbildung 45: Sprachauswahl und Benutzerprofil der Hauptseite

In der Sprachauswahl erfolgt die Anzeige mittels eines *OverlayPanel*, in welchem sämtliche verfügbaren Sprachen *languages* mit zugehöriger Flagge von *country-flag* und Bezeichnung aufgeführt werden. Durch einen weiteren Klick wird *selectLanguage* ausgeführt.

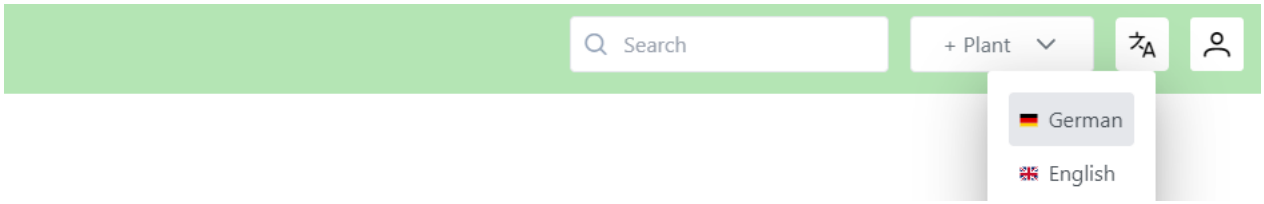


Abbildung 46: Sprachauswahl der Hauptseite

```
34  <OverlayPanel ref="languagePanel" appendTo="body">
35      <div v-for="language in languages" :key="language.code"
36           @click="selectLanguage(language)" class="flex items-center
37           space-x-2 p-2 rounded hover:bg-gray-200 transition duration-300">
38      <country-flag :country='language.code' size='small'/>
39      <span class="me-2 cursor-pointer">{{ language.name }}</span>
40    </div>
41  </OverlayPanel>
```

Abbildung 47: Sprachauswahl der Hauptseite

Innerhalb dem *OverlayPanel* des Benutzerprofils besteht die Möglichkeit, entweder die Einstellungen mittels *showSettings* zu öffnen oder sich mittels *router* abzumelden, um zur Startseite zu gelangen. Die Einstellungen werden demnach in einem *Dialog* geöffnet, welcher bislang noch nicht implementiert wurde.

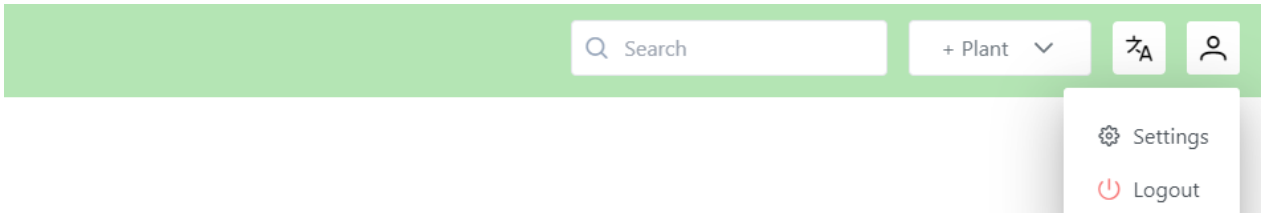


Abbildung 48: Benutzerprofil der Hauptseite

```
  <OverlayPanel ref="settingsPanel" appendTo="body">
    <div class="flex flex-col">
      <div class="flex items-center p-2 rounded hover:bg-gray-200 transition
                  duration-300 cursor-pointer" @click="showSettings = true">
        <i class="pi pi-cog pi-fw me-2"/> Settings
      </div>
      <div class="flex items-center p-2 rounded hover:bg-gray-200 transition
                duration-300 cursor-pointer"  @click="router.push('/start')">
        <i class="pi pi-power-off pi-fw text-red-400 me-2"/> Logout
      </div>
    </div>
  </OverlayPanel>
  <!-- TODO: Settings -->
  <Dialog v-model:visible="showSettings" modal :draggable="false"
          header="Settings" :style="{ width: '33vw' }">
  </Dialog>
```

Abbildung 49: Benutzerprofil der Hauptseite

Variablen wurden verwendet, um die Sucheingabe *searchInput*, ausgewählten Pflanzen *plantsChosen* sowie verfügbaren Pflanzen *plantsOptions* zu verwalten, ebenso wie eine Referenz *languagePanel* und *settingsPanel* zu den *OverlayPanels*. Dabei erfolgt eine alphabetische Sortierung der verfügbaren Pflanzen, welche zu einem späteren Zeitpunkt vom Backend abgerufen werden. Darüber hinaus erfordern die verfügbaren Sprachen zusätzlich ein *code* für die Flagge sowie ein *name* zur Darstellung.

```
<script setup>
import { ref, watch } from 'vue'
import { useRouter } from "vue-router"

const router = useRouter()
const searchInput = ref('')
const showSettings = ref(false)
const languagePanel = ref(null)
const settingsPanel = ref(null)
const plantChosen = ref([])
const plantOptions = ref([])
plantOptions.value.sort((a, b) => a.name.localeCompare(b.name))
const languages = [
  { code: 'de', name: 'German' },
  { code: 'gbr', name: 'English' }
]
```

Abbildung 50: Variablen des Headers

Die Komponente weist vergleichsweise wenig Logik in ihren Funktionen auf und übermittelt die Daten bei jeder Veränderung der Sucheingabe mittels *watch* oder wenn eine Pflanze in *addPlant* hinzugefügt

wurde, an die übergeordnete Komponente. *WatchEffect* soll sicherstellen, dass in *plantOptions* nur Pflanzen zur Auswahl stehen, welche nicht bereits hinzugefügt wurden. Die tatsächliche Anpassung der ausgewählten Sprache wurde bisher jedoch noch nicht implementiert. Die Funktionen der *OverlayPanels* akzeptieren lediglich einen booleschen Wert und invertieren diesen, so dass, falls es noch nicht angezeigt wird, es nun angezeigt wird, und umgekehrt, wenn es bereits angezeigt wird.

```
75  const emit = defineEmits(["add-plant", "search-plant"])
76  const props = defineProps(['plants', 'plantsByUser'])
77  watch(searchInput, (newValue) => {emit("search-plant", newValue)})
78
79  const showLanguagePanel = () => {languagePanel.value.toggle(event)}
80  const showSettingsPanel = () => {settingsPanel.value.toggle(event)}
81
82  function addPlant() {
83    emit("add-plant", plantChosen.value)
84    plantChosen.value = []
85  }
86
87  watchEffect(() => {
88    if (props.plants.length > 0) {
89      plantOptions.value = [...props.plants]
90        .filter(plant => !props.plantsByUser
91        .some(userPlant => userPlant.ID === plant.ID))
92        .sort((a, b) => a.name.localeCompare(b.name))
93    }
94  })
95
96  function selectLanguage(language) {
97    //TODO: Languages
98  }
99  </script>
```

Abbildung 51: Funktionen des Headers

3.2.2.2. Filter (Component)

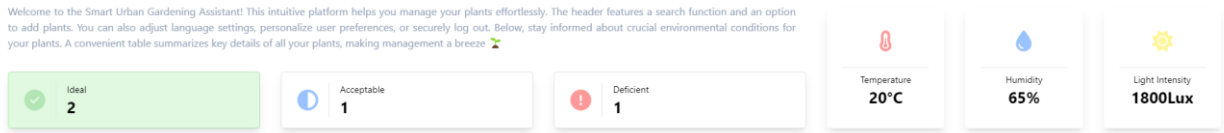
Die linke Seite soll zwei Drittel des verfügbaren Raumes beanspruchen, wobei zuerst eine Beschreibung der Hauptseite angezeigt wird.

```
1  <template>
2      <div class="flex">
3        <div class="lg:w-[66%]">
4          <div class="text-slate-400 p-4 overflow-y-auto text-justify mx-4">
5            Welcome to the Smart Urban Gardening Assistant! ...
6          </div>
```

Abbildung 52: Beschreibung der Hauptseite

Danach folgt eine interaktive Karte, gleichförmig auf dem verbleibenden Raum verteilt, auf welcher der Status der Pflanzen als Filteroption *filterOptions* möglich ist. Es werden ein passendes Icon, welches die Farbe des Status besitzt, der Status selbst und die Anzahl der Pflanzen mit diesem Status angezeigt. Nach Aktivierung eines Filters wird der Hintergrund entsprechend farblich hervorgehoben und es werden nur Pflanzen mit demselben Status in der Pflanzentabelle aufgelistet. Falls man wieder

alle Pflanzen unabhängig von Status anzeigen will, genügt ein erneuter Klick auf die Karte. *Index* des ausgewählten Filters wird in *filterChosen* gespeichert, und nach einem Klick wird *toggleFilter* aufgerufen. Die verwendeten Icons stammen aus *font-awesome-icon*, welche innerhalb *client-only* gezeichnet werden, um Warnungen bezüglich *hydration node mismatch* zu vermeiden.




```
7         <div class="flex lg:flex-row flex-col ms-8 lg:ms-4">
8             <div v-for="(card, index) in filterOptions"
9                  :key="index" class="flex-1 lg:mx-4 my-4">
10              <Card class="flex flex-row rounded py-4 px-6" shadow hover
11                    :class="{ 'bg-[#E0F9E0]': filterChosen === index }"
12                    @click="toggleFilter(index)">
13                <div class="flex items-center">
14                  <client-only>
15                     <font-awesome-icon :icon="card.icon" class="text-[32px]"
16                                        style="{ 'color': card.iconColor }"/>
17                  </client-only>
18                  <div class="mx-4 h-full border-l border-gray" />
19                  <div class="flex flex-col">
20                    <div class="text-sm">{{ card.name }}</div>
21                    <div class="text-2xl font-bold">{{ card.count }}</div>
22                  </div>
23                </div>
24              </Card>
25            </div>
26          </div>
27        </div>
```

Abbildung 53: Filter der Hauptseite

Das restliche Drittel des verbleibenden Raumes wird zur Darstellung der erfassten Umweltbedingungen *environmentalConditions* aus dem Embedded System genutzt. Diese werden in der Desktopansicht nebeneinander und in der Smartphoneansicht untereinander angeordnet.

```
28      <div class="lg:w-[33%] flex lg:flex-row flex-col mb-4">
29          <div v-for="(card, index) in props.environmentalConditions"
30               :key="index" class="flex-1 mx-4 mt-6">
31            <Card class="flex flex-col justify-center text-center bg-white
32                          rounded py-4 px-6 h-full" shadow>
33              <client-only>
34                 <font-awesome-icon :icon="card.icon" class="text-[32px] mb-4"
35                                    :style="{'color': card.iconColor }"/>
36              </client-only>
37              <div class="border-b border-gray my-2"></div>
38              <div class="text-sm">{{ card.name }}</div>
39              <div class="text-2xl font-bold">
40                   {{ card.value }}{{ card.unit }} </div>
41            </Card>
```

Abbildung 54: Umweltbedingungen der Hauptseite

Es werden zwei Datensätze von der übergeordneten Komponente empfangen: Die Liste der Pflanzen *plants* wird genutzt, um die Anzahl der Pflanzen nach ihrem jeweiligen Status zu ermitteln. Gleichzeitig werden die Umweltbedingungen *environmentalConditions* übermittelt, um sie darzustellen. Wenn *filterChosen* wie zu Beginn den Wert -1 aufweist, signalisiert dies eine deaktivierten Filter. Die *filterOptions* beinhalten Informationen zum Status, zur Anzahl der Pflanzen und das zugehörige Icon mit der passenden Farbe.

```
<script setup>
import { ref, watch, defineProps } from 'vue'

const props = defineProps(['plants', 'environmentalConditions'])

const filterChosen = ref(-1)
const filterOptions = ref([
  { name: 'Ideal', count: 0,
    icon: ['fas', 'fa-check-circle'], iconColor: '#B4E5B4' },
  ...
])
```

Abbildung 55: Variablen des Filters

Bei einer Änderung von *filterChosen* wird mittels *emit* der Wert des Filters an die übergeordnete Komponente gesendet, um die entsprechenden Pflanzen in der Tabelle anzuzeigen. Dabei wird ebenso abgefragt, ob der neue Wert dem vorherigen entspricht. Dabei wird ebenfalls geprüft, ob der neue Wert dem vorherigen entspricht. Wenn ja, bedeutet dies, dass der bereits aktive Filter erneut angeklickt wurde. In diesem Fall wird der Filter deaktiviert und auf den Wert -1 zurückgesetzt. Andernfalls wird *filterChosen* auf den Wert des neuen Filters aktualisiert. Innerhalb *updateFilter* werden die Anzahl der Pflanzen für jeden Status ermittelt. Diese Funktion wird jedes Mal aufgerufen, wenn die Hauptseite geladen wird oder sich die Pflanzen in der übergeordneten Komponente verändern, beispielsweise durch Hinzufügen oder Entfernen von Pflanzen.

```
const emit = defineEmits(["filter-plant"])
watch(filterChosen, (newValue) => {emit("filter-plant", newValue)})
const toggleFilter = (index) => filterChosen.value =
                                filterChosen.value === index ? -1 : index

const updateFilter = () =>
  filterOptions.value.forEach((option) => option.count =
        props.plants.filter((plant) => plant.status === option.name).length)

onMounted(() => {updateFilter()})
watch(() => props.plants, () => {updateFilter()}, { deep: true })
</script>
```

Abbildung 56: Funktionen des Filters

3.2.2.3. Hauptseite (Page)

Die Integration der beiden vorherigen Komponenten, nämlich *Header* und *Filter*, erfolgt zuerst auf der Hauptseite. Hierbei werden sämtliche *emits* zu einer Funktion umgeleitet, wobei alle entsprechenden *props* übergeben werden.

```
<template>
    <Header @add-plant="addPlant" @search-plant="searchPlant"
            :plants="plants" :plantsByUser="plantsByUser"/>
    <Filter @filter-plant="filterPlant" :plants="plants"
            :environmental-conditions="environmentalConditions"/>
```

Abbildung 57: Header und Filter in Hauptseite

Die Darstellung der Pflanzen *filterPlants* vereint simultan diverse Funktionen. In erster Linie ermöglicht sie die Anpassung der Spaltenbreite mittels *resizable-columns*. Darüber hinaus gewährleistet die Anzeige mittels *pagination* eine strukturierte Aufteilung in Seiten, um auch bei einer beträchtlichen Anzahl von Pflanzen die Übersicht zu bewahren. Hierbei steht die Auswahl zwischen 5, 10 oder 25 Zeilen *rowsPerPage* zur Verfügung, wobei anfangs 5 *rows* ausgewählt ist. Weiters besteht die Option, zwischen den Seiten vorwärts oder rückwärts zu navigieren sowie zum Anfang oder Ende der Seiten zu springen. Zuletzt bietet es die Funktionalität Zeilen aufzuklappen, welche in *expandedRows* verwaltet werden und *onRowExpand* aufrufen, um weitere Informationen anzuzeigen. Hierbei umfasst *slotProps.data* innerhalb *expansion* die Informationen einer einzelnen Zeile, folglich einer Pflanze.

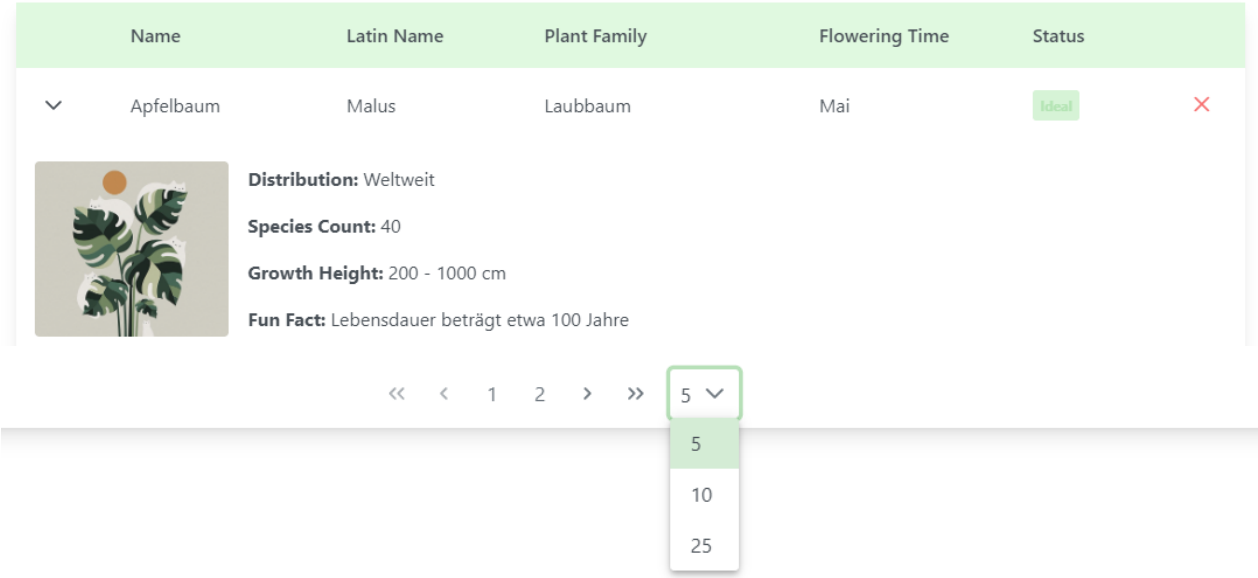


Abbildung 58: Pflanzentabelle der Hauptseite

```
        <DataTable :value="filterPlants" v-model:expandedRows="expandedPlants"
        @rowExpand="onRowExpand" paginator :rowsPerPageOptions="[5,10,25]"
        :rows="5" resizable-columns class="mt-8 mx-8 rounded shadow-lg">
        <Column expander/>
         <template #expansion="slotProps">
          <div class="flex items-center">
            <div class="flex-1"><strong>{{ $t('distribution') }}:</strong>
```

```
                                {{ slotProps.data.distribution }}</div>
            <div class="flex-1"><strong>{{ $t('speciesCount') }}:</strong>
                               {{ slotProps.data.speciesCount }}</div>
            <div class="flex-1"><strong>{{ $t('growthHeight') }}:</strong>
                               {{ slotProps.data.growthHeightMin }} -
                               {{ slotProps.data.growthHeightMax }} cm</div>
            <div class="flex-1"><strong>{{ $t('funFact') }}:</strong>
                               {{ slotProps.data.funFact }}</div>
          </div>
        </template>
```

Abbildung 59: Pflanzentabelle der Hauptseite

Der Status wird nicht lediglich textuell dargestellt, sondern vielmehr visuell mittels Implementierung adäquater Farbgebung, welche dem spezifischen Status entspricht, hervorgehoben. Auf der rechten Seite der Tabelle findet sich ein *Icon*, welches es ermöglicht, Pflanzen aus der Tabelle zu entfernen. Hierbei wird *ConfirmPopup confirmDeletion* eingesetzt, um sicherzustellen, dass die Entfernung beabsichtigt ist. Nach erfolgter erneuter Bestätigung erfolgt die Löschung der betreffenden Pflanze.

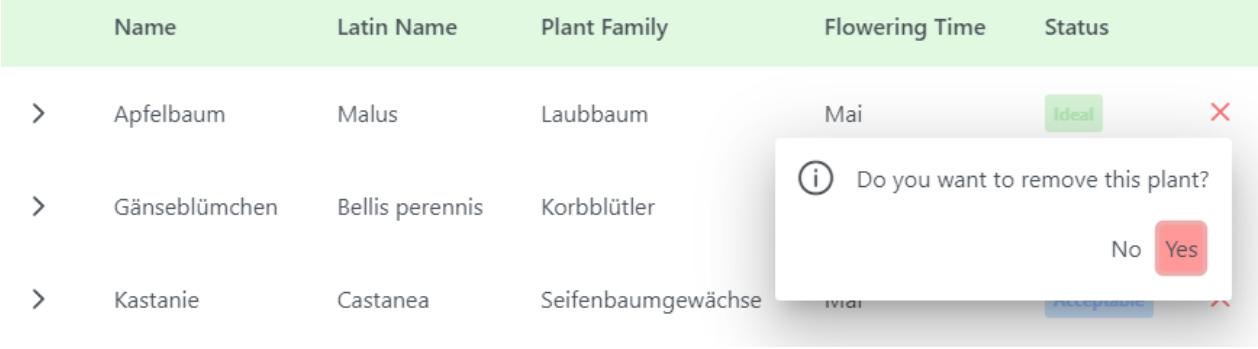


Abbildung 60: Status und Pflanze löschen in der Hauptseite

```
        <Column field="name" header="Name"/>
        <Column field="latinName" header="Latin Name"/>
        <Column field="plantFamily" header="Plant Family"/>
        <Column field="floweringTime" header="Flowering Time"/>
        <Column field="status" header="Status">
          <template #body="slotProps">
            <Tag :style="{ color: slotProps.data.status === 'Ideal' ? '#B4E5B4':
            (slotProps.data.status === 'Acceptable' ? '#99C2FF' : '#FF9999'),
            backgroundColor: slotProps.data.status === 'Ideal' ? '#D6F3D6' :
            (slotProps.data.status === 'Acceptable' ? '#CCE5FF' : '#FFCCCC') }">
            {{slotProps.data.status}}
            </Tag></template>
        </Column>
        <Column>
          <template #body="slotProps">
              <i class="pi pi-times text-red-400 cursor-pointer"
                 @click="confirmDeletion($event, slotProps.data)"/></template>
        </Column>
      </DataTable>
      <ConfirmPopup/>
</template>
```

Abbildung 61: Pflanzentabelle der Hauptseite

Nun werden sämtliche Variablen initialisiert, welche entweder mittels *props* in anderen Komponenten verwendet oder mittels *emit* an diese übergeordnete Komponente übertragen wurden. *Plants* und *environmentalConditionByPlant* werden später aus dem Backend abgerufen, während value von *environmentalConditions* ebenfalls später vom Embedded System bereitgestellt wird.

```
52  <script setup>
53  import { ref, computed } from 'vue'
54  import { useConfirm } from "primevue/useconfirm"
55  import Header from "../components/Header.vue"
56  import Filter from "../components/Filter.vue"
57
58  const searchInput = ref('')
59  const plants = ref([])
60  const expandedPlants = ref([])
61  const environmentalConditions = ref([
62        { name: 'Temperature', unit: '°C', value: 20,
63          icon: ['fas', 'thermometer-half'], iconColor: '#FF9999' },
64        ...])
65  const environmentalConditionByPlant = ref([])
```

Abbildung 62: Variablen der Hauptseite

In *filterPlants* werden die gefilterten Pflanzen für die Tabelle gespeichert. Dies umfasst einerseits searchInput aus der Sucheingabe im *Header*, den ausgewählten Status des *Filters* sowie die alphabetische Sortierung, sodass die gleichzeitige Nutzung sämtlicher Funktionen eine effiziente Suche nach Pflanzen ermöglicht.

```
65  const filterChosen = ref(-1)
66  const filterPlant = (newValue) => filterChosen.value = newValue
67  const searchPlant = (newValue) => searchInput.value = newValue
68  const filterPlants = computed(() => {
69    return plants.value
70      .filter((plant) => {
71        return (
72          plant.name.toLowerCase().includes(searchInput.value.toLowerCase()) &&
73          ((filterChosen.value === -1) ||
74          (filterChosen.value === 0 && plant.status === 'Ideal') ||
75          (filterChosen.value === 1 && plant.status === 'Acceptable') ||
76          (filterChosen.value === 2 && plant.status === 'Deficient'))
77        )
78      }).sort((a, b) => a.name.localeCompare(b.name)) })
```

Abbildung 63: Gefilterte Pflanzen der Hauptseite

Es werden die aus *Header* stammenden *plantsChosen* entnommen, und sofern die betreffende Pflanze noch nicht in der Liste enthalten ist, erfolgt ihre Hinzufügung in *addPlant*. Falls im *ConfirmPopup* eine erneute Bestätigung vorliegt, wird *accept* innerhalb *confirmDeletion* aufgerufen, um die betreffende Pflanze zu entfernen. Innerhalb *onRowExpand* wird sichergestellt, dass nur eine Zeile gleichzeitig erweitert werden kann. Daher wird die vorherige Zeile eingeklappt, um die Übersicht zu bewahren.

```
81  function addPlant(plantsChosen) {
82    plantsChosen.forEach((plant) => {
83      if (!plants.value.some((p) => p.id === plant.id)) plants.value.push(plant)
84    })
85  }
86
87  const confirm = useConfirm()
88  const confirmDeletion = (event, plant) => {
89      confirm.require({
90          target: event.currentTarget,
91          message: 'Do you want to remove this plant?',
92          icon: 'pi pi-info-circle',
93          acceptClass: 'p-button-danger p-button-sm',
94          accept: () => {
95            const plantIndex = plants.value.findIndex(p => p.id === plant.id)
96            if (plantIndex !== -1) plants.value.splice(plantIndex, 1)
97          }
98      })
99  }
100
101 const onRowExpand = (event) =>
102       expandedPlants.value = event.data ? [event.data] : []
```

Abbildung 64: Hinzufügen und Entfernen von Pflanzen in der Hauptseite

3.2.3. Datenmodell

Ein Entity-Relationship-Diagramm fungiert als instrumentale Darstellungsform, um die Struktur einer Datenbank zu konzipieren und zu modellieren. Dieses Diagramm wird in der Initiierungsphase des Projektrealisierungsprozesses erstellt, um eine präzise und kohärente Vorstellung von den Entitäten (Datenobjekten), ihren Beziehungen sowie den Attributen inklusive ihrer Datentypen zu etablieren.

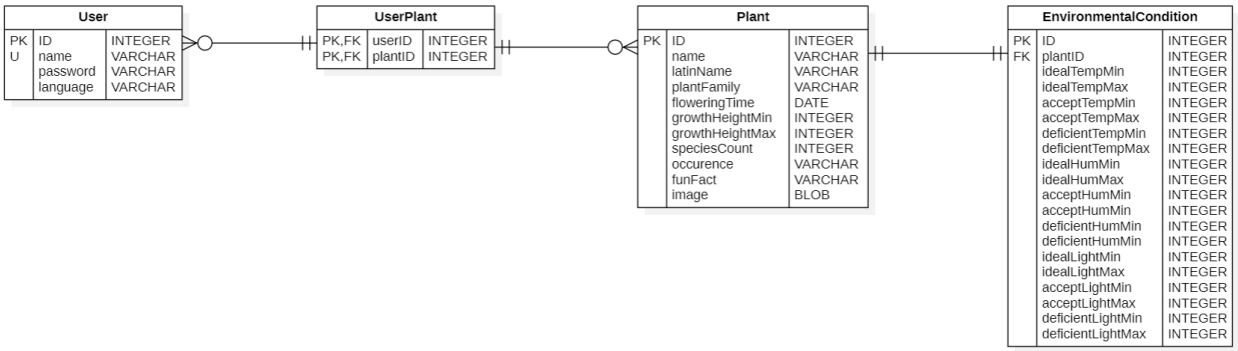


Abbildung 65: Datenmodell

Entitäten (Tabellen): Manifestation erfolgt mittels rechteckiger Darstellung, welche sich im gegenwärtigen Kontext in *User, Plant, UserPlant* und *EnvironmentalCondition* konkretisiert.

Attribute (Felder): Darstellung findet innerhalb der Entitäten statt, wie beispielsweise *ID, username, temperatureMin* und dergleichen.

Primärschlüssel (Primary Key): Kennzeichnet sich mittels der Eigenschaft *PK* des entsprechenden Attributes aus. Der Primärschlüssel fungiert als eindeutiger Identifikator für jede Zeile in der Tabelle.

Fremdschlüssel (Foreign Key): Illustriert die Beziehungen zwischen verschiedenen Tabellen. Der Fremdschlüssel in einer Tabelle stellt eine Referenz auf den Primärschlüssel einer anderen Tabelle dar, was mithilfe der Kennzeichnung *FK* erkannbar ist.

Unique Constraint (Unique): Gibt an, dass der Wert in diesem Feld ein Unikat ist und keine Wiederholung zulässt.

Datentypen (Varchar, Integer, Date, Blob): Indizieren den Typ der Daten, welche in einem bestimmten Attribut gespeichert werden. *Varchar* steht hierbei für Zeichenketten, *Integer* für ganze Zahlen, *Date* für Datumsangaben und *Blob* für Binärdaten von Bildern.

3.2.3.1. User

Für die einschlägigen Benutzerdaten wurde eine Tabelle *user* erstellt. Diese Tabelle verfügt über einen Primärschlüssel *PK* mit dem Bezeichner *ID* sowie über die Attribute *name* und *password* für die Anmeldungs- bzw. Registrierungsinformationen. Darüber hinaus wird in *language* die vom Nutzer gewählte Sprache gespeichert.

3.2.3.2. Plant

Die Tabelle *plant* fungiert als Speicherort für Informationen über Pflanzen mit passenden Attributen wie *PlantFamily* oder *FloweringTime*. Jede Pflanze ist zudem mit bestimmten Umweltbedingungen verknüpft, welche ihren optimalen oder suboptimalen Wachstumsbedingungen entsprechen. Mittels einer One-to-One-Beziehung sind beide Tabellen miteinander verbunden, wodurch jeder Tabelle der Primärschlüssel der anderen als Fremdschlüssel zugeordnet ist.

3.2.3.3. Userplant

Um den Benutzern die Möglichkeit zu geben, eine oder mehrere Pflanzen zu besitzen, wird die Many-to-Many-Beziehung mittels einer Zwischentabelle realisiert. In dieser sind die Primärschlüssel *UserID* und *PlantID* aus den entsprechenden Tabellen aufgeführt, wobei sie sowohl als Primärschlüssel als auch als Fremdschlüssel definiert werden.

3.2.3.4. EnvironmentalCondition

Die Speicherung der idealen, akzeptablen und mangelhaften minimalen sowie maximalen Werte für Temperatur, Luftfeuchtigkeit und Lichtstärke einer Pflanze erfolgt in einer selbstständigen Tabelle. Dies ermöglicht es den Benutzern, diese Werte für ihre Pflanzen zu überprüfen und bei Bedarf zu optimieren. Dabei wird die *PlantID* als Fremdschlüssel markiert, um eine direkte Zuordnung zu ermöglichen.

3.3. Meilenstein 3

3.3.1. Vollendung des Frontends

In diesem Kapitel werden lediglich jene Codefragmente präsentiert, welche als ergänzende Elemente zum zweiten Meilenstein hinzugefügt wurden und somit ausschließlich im Kontext desselben ihre Vollständigkeit erlangen.

3.3.1.1. Startseite

Mittels *watch* erfolgt die fortlaufende Überwachung von *username* und *password*, wobei eine Änderung stets eine Ausführung initiiert. Somit wird *disabledButton* erst *false* gesetzt, wenn beide Eingabefelder einen Wert aufweisen, anderenfalls bleibt der *Button* deaktiviert.

```
1  <Button @click="login" :disabled="disableButton">
2  const disableButton = ref(true)
3  watch([username, password], () =>
4  disableButton.value = !username.value || !password.value)
```

Abbildung 66: Aktivierung des Buttons bei Eingabe

Darüber hinaus soll gewährleistet werden, dass ein Benutzer keinesfalls unangemeldet auf die Hauptseite gelangen kann, etwa mittels Verwendung der URL. Infolgedessen soll im *store* der angemeldete Benutzer persistiert werden, um einen globalen Zugriff von allen Seiten aus zu ermöglichen. Vuex ist hierbei ein zentrales State-Management-Tool, was die Verwaltung von Zuständen ermöglicht. Dazu ist die Ausführung des folgenden Befehls und die Erstellung eines entsprechenden Verzeichnisses mit einer Datei *stores/index.js* erforderlich.

```
> npm install vuex
```

```
1  import { createStore } from 'vuex'
2
3  export default createStore({
4    state: {
5      auth: {
6        user: null
7      }
8    },
9    mutations: {
10     setUser(state, user) {
11       state.auth.user = user
12     }
13   },
14   getters: {
15     getUser: state => state.auth.user
16   }
17 })
```

Abbildung 67: stores/index.js

Letztlich muss eine Verbindung zur App unter plugins/index.js hergestellt werden.

```
1  import stores from '../stores'
2  nuxtApp.vueApp.use(stores)
```

Abbildung 68: stores in plugins/index.js

Mittels *store.commit* kann nun die Funktion *setUser* aufgerufen und *user* als Parameter übergeben werden, um in der Funktion *login* der Startseite den angemeldeten Benutzer zu persistieren.

```
import { useStore } from 'vuex'
const store = useStore()
store.commit('setUser', user)
```

Abbildung 69: Benutzer persistieren

Letztlich kann man in *onMounted* der Hauptseite gleich anfangs mittels *store.getters* die Funktion *getUser* aufrufen, und falls kein *user* zurückgeliefert werden sollte, erfolgt eine Weiterleitung des Benutzers zur Startseite, um sich anzumelden.

```
onMounted(() => if (!store.getters.getUser) router.push('/start'))
```

Abbildung 70: Benutzer erhalten

3.3.1.2. Sprachauswahl

Um eine mehrsprachige Webanwendung anzubieten, sollten sämtliche verwendeten Texte in einer JSON-Datei unter *locales* organisiert werden: *de.json* auf der linken Seite und *en.json* auf der rechten Seite. *Placeholder* wurde hierbei als beispielhafte Darstellung verwendet.

```
{
 "placeholder": "Text einfügen..."
}
```

```
{
 "placeholder": "Insert text..."
}
```

Abbildung 71: locales

Mittels *vue-i18n* wird die Verwaltung verschiedener Sprachen ermöglicht. Hierfür ist es zunächst nötig eine Verbindung zur App unter *plugins/index.js* herzustellen, bei der entsprechende JSON-Dateien übergeben und Englisch als die voreingestellte Standardsprache definiert werden.

```
import { createI18n } from 'vue-i18n'
let messages = Object.fromEntries(
      Object.entries(import.meta.globEager('../locales/*.json'))
      .map(([key, value]) => {return [key.slice(11, -5), value.default]}))
const i18n = createI18n({
    legacy: false,
    locale: 'en',
    messages: messages,
})

nuxtApp.vueApp.use(i18n)
```

Abbildung 72: Konfiguration von vue-i18n

Letztlich kann in *Header* die Sprache mittels *locale* festgelegt werden.

```
const { locale } = useI18n()
function selectLanguage(language) {
  locale.value = language.locale
  showLanguagePanel()
}
```

Abbildung 73: Sprachauswahl

Durch die Verwendung von *$t* besteht nun die Möglichkeit, den Schlüssel *placeholder* einzufügen und abhängig von der Sprache den entsprechenden Wert anzuzeigen.

```
<div>{{ $t(placeholder) }}</div>
```

Abbildung 74: Anwendung von locales

3.3.1.3. Benutzereinstellungen

Die Eingabefelder ähneln denen auf der Startseite. Jedoch wird hierbei eine Überprüfung durchgeführt, um sicherzustellen, dass das alte Passwort korrekt erneut eingegeben wurde, das neue Passwort nicht mit dem alten Passwort übereinstimmt und die erneute Eingabe des Passworts übereinstimmt. Erst wenn alle drei Bedingungen erfüllt wurden, kann man das Passwort aktualisieren. Darüber hinaus besteht die Möglichkeit, den Account zu löschen, ähnlich wie bei den Pflanzen, bei denen ebenfalls nach einer erneuten Bestätigung gefragt wird.

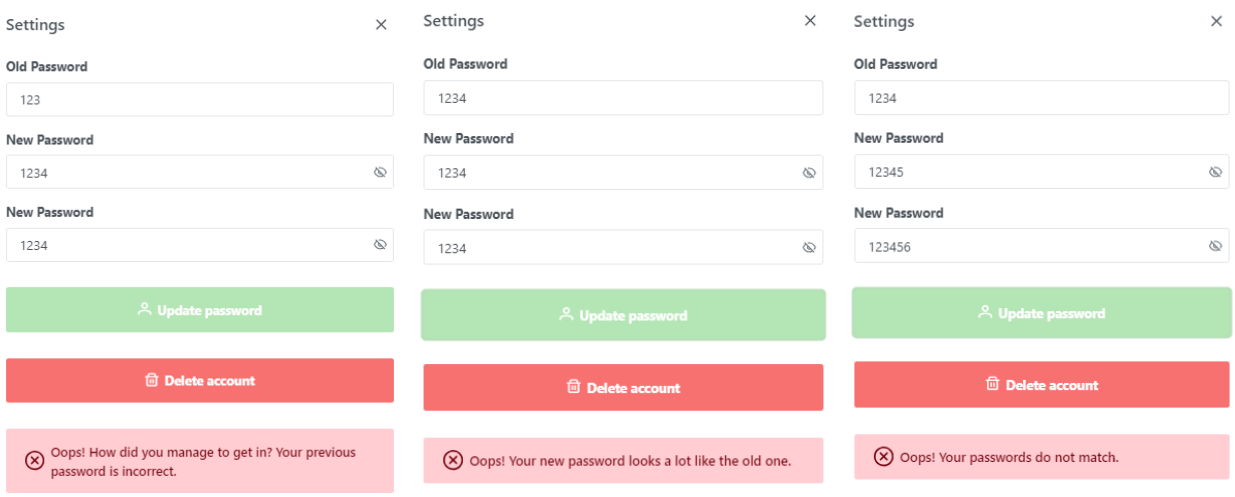


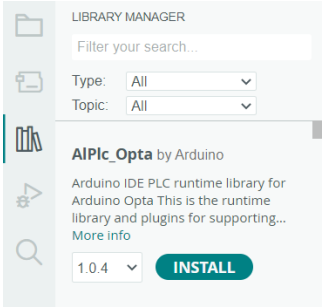
Abbildung 75: Benutzereinstellungen

```
<Message v-if="wrongOldPassword" severity="error"
:closable="false"> {{ $t('wrongOldPassword') }} </Message>
<Message v-else-if="newPasswordLikeOldPasswort" severity="error"
:closable="false"> {{ $t('newPasswordLikeOldPasswort') }} </Message>
<Message v-else-if="newPasswordsNotMatching" severity="error"
:closable="false"> {{ $t('newPasswordsNotMatching') }} </Message>
function updatePassword() {
  wrongOldPassword.value = oldPassword.value !==store.getters.getUser.password
  newPasswordLikeOldPasswort.value = oldPassword.value === newPassword.value
      || oldPassword.value === newPasswordRepeat.value
  newPasswordsNotMatching.value = newPassword.value !==newPasswordRepeat.value

if (wrongOldPassword && newPasswordLikeOldPasswort && newPasswordsNotMatching)
//TODO: update password
}
```

Abbildung 76: Benutzereinstellungen

3.3.2. Lichtstärke



Aufgrund der starken Community von Arduino stehen für zahlreiche Aktoren, Sensoren und andere unabdingbare Komponenten Bibliotheken zur Verfügung. Diese erweisen sich als äußerst hilfreich bei der Vereinfachung sowohl der Handhabung als auch der Implementierung. Es genügt, diese Bibliotheken mittels des Library Managers in der Arduino IDE mühelos einzubinden.

Abbildung 77: Arduino Library Manager

Zunächst erfolgte eine erneute Verdrahtung der Pins des BH1750 mit dem Arduino. Danach wurde ein entsprechendes Programm implementiert, wobei die Bibliothek *BH1750 von Christopher Laws* eingebunden wurde. Grundlegend kommuniziert der Mikrocontroller mittels eines Bussystems mit dem Lichtstärkesensor. Daher wurde zusätzlich die Standardbibliothek *Wire.h* inkludiert, um eine erfolgreiche Verbindung herzustellen und das Gerät sachgemäß konfigurieren zu können.

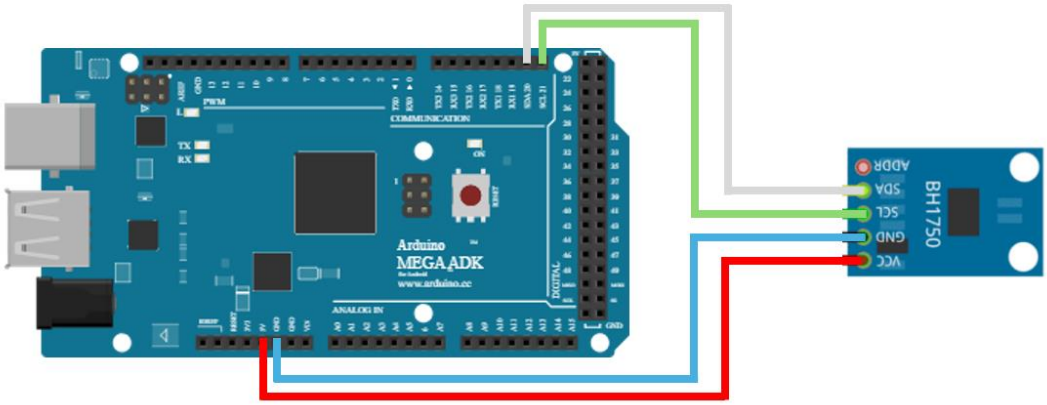


Abbildung 78: Verkabelung des BH1750

```
#include <Wire.h>
#include <BH1750.h>
BH1750 lightMeter;
void setup() {
   Serial.begin(9600);  //Start der seriellen Kommunikation
   Wire.begin();  //Initialisierung der Kommunikation mit dem Sensor
   lightMeter.begin();  //Initialisierung des BH1750
}
void loop() {
   //Messung einlesen und in einer Variable speichern
   float lichtstaerke = lightMeter.readLightLevel();
   Serial.println("Lichtstärke: " +
           String(lichtstaerke) + " lx");
   delay(4000);  //Wartezeit von 4 Sekunden
}
```

Abbildung 79: Programm des BH1750

3.3.3. Projekteinrichtung des Backends

Die vorliegende Serverstruktur folgt einer typischen REST-API-Konfiguration, welche weit verbreitet ist und für moderne Webanwendungen verwendet wird. So wird eine klare Trennung von Server (Backend zur Datenverwaltung) und dem Client (Frontend der Webanwendung) vorgesehen. Dementsprechend wurde ein Ordner *server* auf höchster Ebene im Projekt erstellt. Um jedoch eine Verbindung zur Datenbank realisieren zu können, muss MySQL auf dem Rechner installiert sein. Danach muss ebenso eine Systemumgebungsvariable mit einem Verweis zum *bin* Ordner definiert werden. Folglich kann im Terminal eine Verbindung mittels

mysql -h 127.0.0.1 -P 3306 -u root -p

getestet werden. Bei erfolgreichem Aufbau wird mittels

create database db;

ein tatsächliches Datenbank-Image erstellt, welches im späteren Verlauf die Tabellen und Einträge speichert.

ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'pw123'

ermöglicht einen Zugriff mittels Authentifizierung von anderen Diensten, wie beispielsweise der Entwicklungsumgebung. Darüber hinaus wurde in der Entwicklungsumgebung Visual Studio Code die *mysql2* Erweiterung vorausgesetzt, welche mit *npm install mysql2* eingebunden wurde, um eine Handhabung zu ermöglichen. Weiters ist *SQLTools* die Erweiterung, welche als Client fungierte und somit jegliche Abfragen oder Informationen zur Datenbank erheblich vereinfachte.

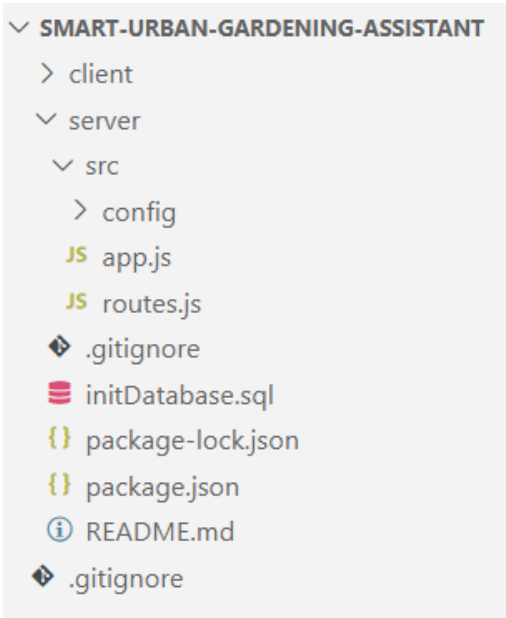


Abbildung 80: Verzeichnisstruktur des Servers

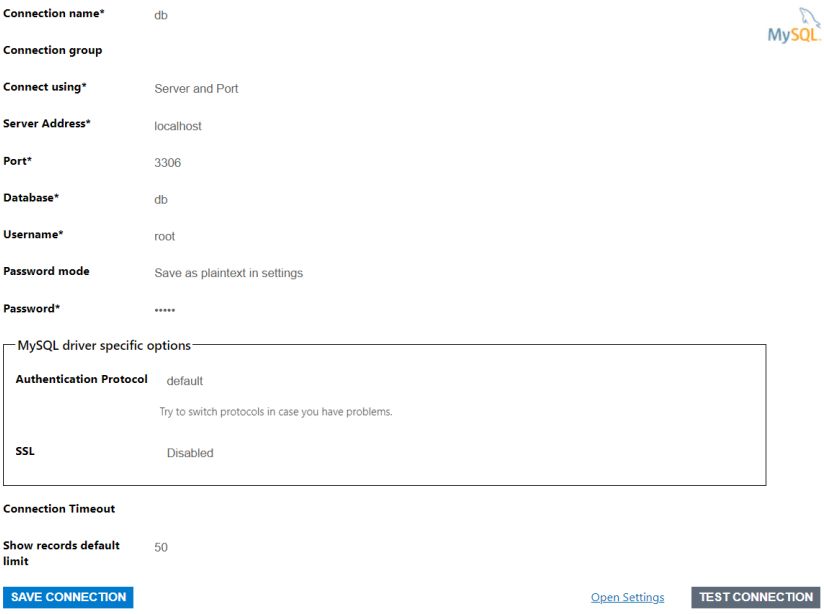


Abbildung 81: Datenbankzugriff über SQLTools

Um im Anschluss eine tatsächliche Verbindung aufbauen zu können, wurden in *config.js* Definitionen der Zugriffs- und Verbindungskonfiguration, wie *Port, Host, User, Passwort* und *Name* für die Datenbank definiert.

```
module.exports = {
  port: process.env.PORT || 3001,
  db: {
    host: process.env.DB_HOST || 'localhost',
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASS || 'pw123',
    database: process.env.DB_NAME || 'db'
  },
  authentication: {
    jwtSecret: process.env.JWT_SECRET || 'secret'
  }
}
```

Abbildung 82: config.js

Database.js stellt die Datenbankverbindung her, indem es einen *pool* von Verbindungen über das *mysql* erstellt. *Pool* wird so exportiert, dass andere Teile der Anwendung Anfragen an die Datenbank senden können.

```
const mysql = require('mysql2')
const config = require('./config')
const pool = mysql.createPool(config.db)

module.exports = pool
```

Abbildung 83: database.js

App.js initialisiert die *express* und konfiguriert Middlewaresysteme wie *cors* und *morgan*. Zusätzlich werden hier die Routen der Anwendung verbunden, der Server auf dem in *config.js* definierten Port gestartet und festgelegt, von welcher Adresse die Anfragen angenommen und bearbeitet werden.

```
const express = require('express')
const cors = require('cors')
const morgan = require('morgan')
const config = require('./config/config')
const routes = require('./routes')

const app = express()
const corsOptions = {
  origin: 'http://localhost:3000',
  methods: 'GET,HEAD,PUT,PATCH,POST,DELETE',
  credentials: true,
  optionsSuccessStatus: 204
}

app.use(cors(corsOptions)).use(morgan('combined'))
   .use(express.json()).use('/', routes)
app.listen(config.port, () => console.log(`Server on port ${config.port}`))
```

Abbildung 84: app.js

Da die Datenbank nur lokal auf jedem Rechner läuft, müssten die Tabellen und Einträge jedes Mal aufs Neue angelegt werden, weswegen ein Dump *initDatabase.sql* erstellt wurde, welcher nach einem einmaligen Durchführen sämtliche Tabellen und die gesamte Datenbankstruktur anlegt.

```sql
CREATE TABLE `user` (
  `ID` int NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `language` varchar(2) NOT NULL,
  PRIMARY KEY (`ID`)
)
```

Abbildung 85: Ausschnitt aus initDatabase.sql

Die Kommunikation zwischen Frontend und Backend in der Anwendung erfolgt hauptsächlich über HTTP-Anfragen, welche mittels *axios* gesendet werden. *Axios* ist eine Bibliothek, welche zum Senden von asynchronen HTTP-Anfragen an REST-Endpoints verwendet wird, um CRUD-Operationen durchzuführen. Installiert wird diese Bibliothek über *npm install axios*. In Webanwendungsarchitekturen ist es üblich, dass der Client (Frontend) und der Server (Backend) auf unterschiedlichen Ports laufen. Dies wird aus mehreren Gründen gemacht, wie beispielsweise zur klaren Trennung von Anliegen, Konfliktvermeidung und einfachere Bereitstellung von Informationen. Hier läuft der Client auf dem Standardport 3000 für lokale Entwicklungsserver im Nuxt.js-Framework und der Node.js-Server auf Port 3001. Der Server wird über *npm start* gestartet und wartet anschließend auf Anfragen.

3.3.4. GET – Endpoints

3.3.4.1. User

Dieser Endpoint stellt eine GET-Anfrage dar, mit welchem alle Benutzer aus der Datenbank abgerufen werden können. Der Endpoint ist auf die URL */users* festgelegt, und die zugehörige Funktion ist als *async* markiert, was darauf hinweist, dass asynchroner Code ausgeführt wird, unter Verwendung von Promises. Innerhalb des *try* Blocks befindet sich eine Datenbankabfrage, welche alle Benutzer aus der Tabelle *user* abruft und über *pool.execute* ausführt. Das Ergebnis dieser Abfrage wird in einem Array strukturiert, und die Benutzerdaten werden folglich im JSON-Format an den Client zurückgesendet. Im *catch* Block wird auf mögliche Fehler während der Datenbankabfrage reagiert. Im Fehlerfall wird eine Fehlermeldung in der Konsole protokolliert, und dem Client wird ein HTTP-Statuscode 500 mit einer JSON-Nachricht übermittelt, welche auf interne Serverfehler hinweist.

```go
router.get('/users', async (req, res) => {
  try {
    const [users] = await pool.query('SELECT * FROM User')
    res.json(users)
  } catch (error) {
    res.status(500).json({ error: 'Internal server error' })}
})
```

Abbildung 86: getUsers Endpoint

GetUsers ist der Aufruf des Endpoints, welcher definiert wurde. Hier wird ebenfalls ein *try-catch* Block implementiert, um auf mögliche Fehler während der Ausführung zu reagieren. Die Funktion sendet über *await axios.get()* eine GET-Anfrage an *http://localhost:3001/users*. Das Ergebnis dieser Anfrage wird in *response* gespeichert. Nachfolgend wird der Statuscode der Antwort überprüft. Bei einem Statuscode von 200 wird *users* mit den erhaltenen Daten aktualisiert. Im *catch* wird auf Fehler während der Anfrage reagiert, indem eine Fehlermeldung in der Konsole ausgegeben wird.

```
9   async function getUsers() {
10    try {
11      const response = await axios.get('http://localhost:3001/users')
12      if (response.status === 200) users.value = response.data
13    } catch (error) {
14      console.error('Fehler beim Get der Benutzer:', error)
15    }
16  }
```

Abbildung 87: getUsers Implementierung

3.3.4.2. UserByName

Der Endpunkt zielt darauf ab zu prüfen, ob ein Benutzer mit einem spezifischen Namen in der Datenbank vorhanden ist. Die URL ist mit */users/:name* festgelegt, wobei *:name* als Platzhalter für den Benutzernamen dient, welcher in der URL angegeben wird. Die Funktion extrahiert diesen Benutzernamen *const { name } = req.params* aus den Parametern der Anfrage. Die Datenbankabfrage selektiert alle Benutzer mit dem angegebenen Namen aus der Tabelle. Dabei wird die Platzhalter-Syntax *?* verwendet, um potenzielle SQL Injection-Angriffe zu verhindern. Die Ergebnisse der Abfrage werden in *results* gespeichert. Wenn Benutzer mit dem angegebenen Namen gefunden werden, also *results.length > 0*, wird eine JSON-Antwort mit einem Statuscode 200 und der Nachricht *User exists* zurückgesendet. Andernfalls wird eine JSON-Antwort mit einem Statuscode 404 und der Nachricht *User not found* generiert.

```
1   router.get('/users/:name', async (req, res) => {
2     const { name } = req.params
3
4     try {
5       const [results] =
6              await pool.query('SELECT * FROM User WHERE name = ?', [name])
7       if (results.length > 0) res.status(200).json({ message: 'User found' })
8       else res.status(404).json({ message: 'User not found' })
9     } catch (error) {
10      res.status(500).json({ error: 'Internal server error' })
11    }
12  })
```

Abbildung 88: getUserByName Endpoint

CheckUserExists ist der Aufruf zu *http://localhost:3001/users/username,* und hat das Ziel zu überprüfen, ob ein Benutzer mit dem angegebenen Benutzernamen bereits in der Datenbank existiert, um dem Benutzer zu zeigen, dass ein neues Profil angelegt werden sollte, falls noch kein aktives besteht. Falls der Benutzer existiert, gibt die Methode *true* zurück, andernfalls *false*.

```
13  async function checkUserExists(username) {
14    try {
15      const response =
16            await axios.get(`http://localhost:3001/users/${username}`)
17      return response.status === 200 //Benutzer gefunden
18    } catch (error) { // Benutzer nicht gefunden
19      if (error.response && error.response.status === 404) return false
20      else console.error('Fehler beim Überprüfen des Benutzers:', error)
```

Abbildung 89: checkUserExists Implementierung

3.3.4.3. Plant/EnvironmentalConition

Die Inhalte der Tabelle *plant* werden wie bei *user* über SQL-Abfragen ausgelesen und dem Frontend über GET – Endpoints als JSON-Objekte übergeben. URL ist hierbei *http://localhost:3001/plants*. Um die passende Umweltbedingung anzuzeigen, welche sich aus den gemessenen Werten des Embedded System ergibt, wurde der Endpoint *http://localhost:3001/plants/:plantId/environmental-conditions* definiert, welcher mit der Abfrage *SELECT * FROM EnvironmentalCondition WHERE plantId = ?,* die passenden *EnvironmentalConditions* zu den *Plants* ausliest.

3.3.4.4. UserPlant

Für die Liste der Pflanzen eines angemeldeten Benutzers wird erneut eine SQL – Abfrage *SELECT * FROM UserPlant WHERE userID = ?', [userId]* erstellt, welche die Ergebnisse in dem Hauptfenster der Anwendung anzeigt.

3.3.5. POST – Endpoints

3.3.5.1. User

Ein *User* wird auch hier über eine Abfrage hinzugefügt. Davor werden zunächst alle Inhalte der Tabelle ausgelesen, welche mit dem eingegebenen *name* übereinstimmen. Diese Benutzer werden in *results* gespeichert, mit welcher man folglich die Existenz prüft und über *res.status().json({ message: '' });* Informationen ausgeben kann. Die Sprache wird nach der Registrierung vorerst mit *defaultLanguage* auf Englisch gesetzt. Beim Aufruf der Methode in *Registration.vue* wird mit *router.push('/main')* wieder zur Startseite weitergeleitet und der Benutzer wird gelöscht.

```
1  router.post('/register', (req, res) => {
2    const { name, password, language } = req.body
3
4    pool.query('SELECT * FROM User WHERE name = ?', [name])
5      .then(([results]) => {
6        if (results.length > 0) {
7          // Benutzer existiert bereits
8          res.status(409).json({ message: 'Username already taken' })
```

```
      } else {
        // Benutzer erstellen
        let defaultLanguage = 'en'
        pool.query(
          'INSERT INTO User (name, password, language) VALUES (?, ?, ?)',
          [name, password, language || defaultLanguage]
        )
        .then(() => {res.status(201).json({ message: 'User created' })})
        .catch((error) => {res.status(500).json({ error: 'Database error' })})
      }
    })
    .catch((error) => {
      res.status(500).json({ error: 'Database error' })
    })
})
```

Abbildung 90: register Endpoint

3.3.5.2. UserPlant

Im Wesentlichen erfolgt das Einpflegen einer weiteren *UserPlant* gleichermaßen wie bei *User*. Hierbei werden lediglich für die Datenbank die IDs der beiden Tabellen *User* und *Plant* benötigt, welche erneut mittels einer Übergabe in der Query *[userId, plantId]* übergeben werden. Aufgrund der Option zur Mehrfachauswahl von Pflanzen, welche hinzugefügt werden können, werden diese unter Anwendung einer Schleife *for (const plant of plantChosen.value)* durchlaufen und einzeln gespeichert.

3.3.6. PUT – Endpoint

3.3.6.1. User

Ein Benutzer kann sein Passwort in den Einstellungen ändern. Mithilfe der HTTP-Methode *put* werden über die URL */users/:id/password* im *req.body* das alte *oldPassword* und das neue *newPassword* mitgegeben und durch die *UPDATE* Abfrage in der Datenbank aktualisiert.

```
router.put('/users/:id/password', async (req, res) => {
  const { id } = req.params;
  const { password } = req.body

  try {
    const [user] = await pool.query('SELECT * FROM User WHERE ID = ?', [id])
    if (user.length === 0)
       return res.status(404).json({ message: 'User not found.' })

    await pool.query('UPDATE User SET password = ? WHERE ID = ?',
       [password, id])
    res.json({ message: 'Password updated successfully.' })
  } catch (error) {
    res.status(500).json({ error: 'Internal server error.' })
  }
})
```

Abbildung 91: putPassword Endpoint

Gleichzeitig werden beim Funktionsaufruf unterschiedliche Überprüfungen durchgeführt, um sicherzustellen, dass das alte Passwort korrekt ist, das neue Passwort nicht dem alten entspricht und bei der wiederholten Eingabe identisch ist.

```
if (store.getters.getUser.password !== oldPassword.value) {
    wrongOldPassword.value = true
    return
}

if (newPassword.value !== newPasswordRepeat.value) {
    newPasswordsNotMatching.value = true
    return }

if (newPassword.value === oldPassword.value) {
    newPasswordLikeOldPasswort.value = true
    return }
```

Abbildung 92: updatePassword Implementierung

3.3.7. DELETE – Endpoints

3.3.7.1. Userplant

Der Aufruf dieses Endpoints wird in der Hauptseite innerhalb der Funktion *deleteUserPlant* durchgeführt. In *store.getters.getUser.ID* wird die ID des aktuell eingeloggten Benutzers hinterlegt, über welche.

```
async function deleteUserPlant(plantId) {
  try {
    await axios.delete(`http://localhost:3001/user/
          ${store.getters.getUser.ID}/plants/${plantId}`)
    plantsByUser.value.splice(plantsByUser.value
                        .findIndex(plant => plant.ID === plantId), 1)
  } catch (error) {
    console.error('Fehler beim Delete der Pflanze: ' +  error)
  }
}
```

Abbildung 93: deleteUserPlant Endpoint

Die Funktion *useConfirm()* wird verwendet, um eine Bestätigungsdialogbox zu implementieren. Die Funktion *confirmDeletion* wird aufgerufen, wenn der Benutzer beabsichtigt, eine Pflanze zu löschen. Dabei wird eine Dialogbox angezeigt, die nachfragt, ob er die ausgewählte Pflanze entfernen möchte. Das Dialogfenster enthält eine Meldung, ein Informations-Symbol und einen roten, kleinen *accept* Button, welcher die implementierte Methode *deleteUserPlant(plantID)* aufruft, um die Löschung der Pflanze mit der angegebenen *plantID* durchzuführen.

```
const confirm = useConfirm()
const confirmDeletion = (event, plantID) => {
  confirm.require({
    target: event.currentTarget,
    message: 'Do you want to remove this plant?',
    icon: 'pi pi-info-circle',
    acceptClass: 'p-button-danger p-button-sm',
```

```
      accept: () => {
        deleteUserPlant(plantID)
      }
    })
  }
```

Abbildung 94: confirmDeletion Implementierung

3.3.7.2. User

Das Löschen eines *User* erfolgt prinzipiell identisch wie bei *UserPlant*. Der einzige Unterschied beläuft sich dabei, dass hier sowohl die Pflanzen des Benutzers in *UserPlant* als auch der *User* an sich aus der Datenbank entfernt werden müssen. Dies wurde mittels zwei aufeinanderfolgender Queries gelöst. Ansonsten erfolgt der Aufruf wieder über einen Endpoint *http://localhost:3001/users/:id*

```
1  await pool.execute('DELETE FROM UserPlant WHERE userID = ?', [id])
2  await pool.execute('DELETE FROM User WHERE ID = ?', [id])
```

Abbildung 95: deleteUser Endpoint

3.4. Endabgabe

3.4.1. Kommunikation zwischen Webanwendung und Embedded System

Anfänglich wurde der WLAN-Chip ESP8266 mit dem Arduino verdrahtet. Danach konnte mit der Implementierung der Kommunikation begonnen werden, welche eine Einbindung der Bibliothek *WiFiEsp by bportaluri* voraussetzte. Die Kommunikation geschieht hierbei über die seriellen Kommunikationspins RX und TX des Boards. Daher wurde das Modul angeschlossen und in der *setup()* Methode über *Serial2.begin()* initialisiert.

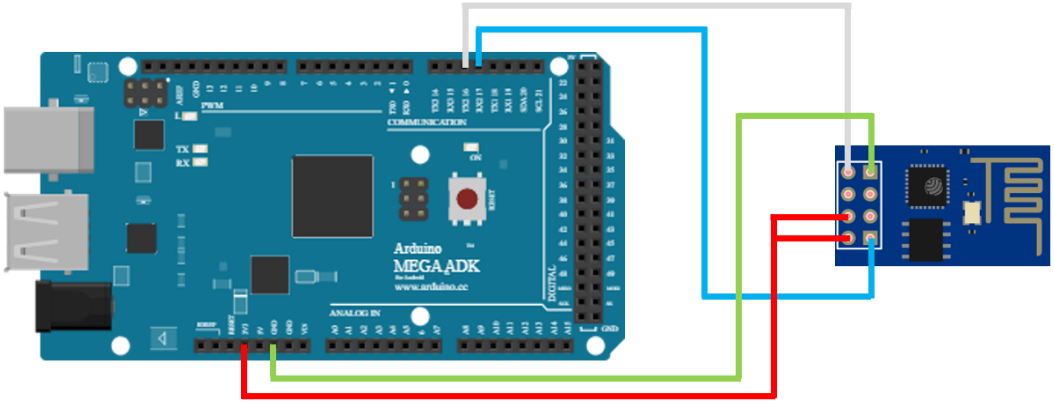


Abbildung 96: Verkabelung des ESP8266

Der ESP8266 dient als Webserver für die Webanwendung, weswegen über *ssid* und *pass* zunächst eine Verbindung zum Modul hergestellt und mittels *port* der Server gestartet wird.

```
#include "WiFiEsp.h"

char ssid[] = "iPhone von Valon";      // Netzwerk SSID (Name)
char pass[] = "12345678";             // Netzwerkkennwort
int status = WL_IDLE_STATUS;         // Status des Wifi-Radios
int port = 80;                              // Port für den Webserver

WiFiEspServer server(port);

void setup() {
  Serial.begin(115200);   // Serial 0 für Debugging
  Serial2.begin(115200);  // Serial 2 für ESP-Modul

  WiFi.init(&Serial2);                                // ESP-Modul
  if (WiFi.status() == WL_NO_SHIELD) {          // Shield vorhanden?
    Serial.println("WiFi shield not present");        // Stopp, wenn nicht
vorhanden
    while (true);
  }
}
```

Abbildung 97: Setup der Kommunikation

Unter Verwendung der Methoden *begin()* wird realisiert, dass die einzelnen Komponenten gestartet werden. Damit der Client den in *processClient()* definierten GET-Endpoint aufrufen kann, muss die IP-Adresse des Geräts im Netzwerk bekannt sein, welche in einer Ausgabe auf dem Serial Monitor angezeigt wird.

```
  while (status != WL_CONNECTED) {
    Serial.print("Attempting to connect to WPA SSID: ");
    Serial.println(ssid);
    status = WiFi.begin(ssid, pass);  // Verbindungsversuch mit Netzwerk
  }
  Serial.println("You're connected to the network");

  server.begin();  // Webserver

  Serial.println();
  printCurrentNet();
  Serial.println();
  printWifiData();
}

void loop() {
  WiFiEspClient client = server.available();// Warte auf Verbindung zum Client

  if (client) {
    Serial.println("New client");
    processClient(client);
    client.stop();
  }

  delay(10000);  // Reduktion der Sensorauslastung
}

void processClient(WiFiEspClient& client) {
  while (client.available()) {                          // Warte auf Anfrage
    String request = client.readStringUntil('\r');      // Anfrage
    if (request.indexOf("GET") != -1) {                 // Anfrage ein GET?
      if (request.indexOf("/environmentalCondition") != -1) {
        // Anfrage zu Umweltbedingungen?
        handleGETRequest(client);
      }
    }
  }
```

Abbildung 98: Loop der Kommunikation

Sobald die Anfrage der Webanwendung beim Webserver eintrifft, werden die Sensordaten *environmentalCondition* in der Methode *handleGETRequest()* ausgelesen, als JSON formatiert und mit einem HTTP-Header zurückgesendet.

```
56  void handleGETRequest(WiFiEspClient& client) {
57    String environmentalCondition = "{ \
58        "temperature\": " + String(temperatur) + ", \
59        "humidity\": " + String(luftfeuchtigkeit) + ", \
60        "lightIntensity\": " + String(lichtstaerke) + " }";
61
62    client.println("HTTP/1.1 200 OK");
63    client.println("Content-Type: application/json");
64    client.print("Content-Length: ");
65    client.println(environmentalCondition.length());
66    client.println("Connection: close");
67    client.println();
68    client.println(environmentalCondition);
69  }
70
71  void printWifiData() {
72    IPAddress ip = WiFi.localIP();
73    Serial.print("IP Address: ");
74    Serial.println(ip);
75
76    byte mac[6];
77    WiFi.macAddress(mac);
78    char buf[20];
79    sprintf(buf, "%02X:%02X:%02X:%02X:%02X:%02X", mac[5], mac[4], mac[3],
80  mac[2], mac[1], mac[0]);
81    Serial.print("MAC address: ");
82    Serial.println(buf);
83  }
84
85  void printCurrentNet() {
86    Serial.print("SSID: ");
87    Serial.println(WiFi.SSID());
88  }
```

Abbildung 99: Funktionen der Kommunikation

Demzufolge ist es erforderlich, die Umweltbedingungen des bereitgestellten Endpoints des Embedded Systems in der Webanwendung mittels *axios* über HTTP zu erfassen. Dies soll durch die Verwendung von *setInterval* in einem Intervall von 60000 Millisekunden erfolgen, um einen minutengenauen Rhythmus zu gewährleisten.

```
1   async function getEnvironmentalCondition(){
2     try {
3         const response =
4               await axios.get('http://192.168.178.31:80/environmentalCondition')
5         environmentalConditions.value[2].value = response.data.lightIntensity
6         environmentalConditions.value[1].value = response.data.humidity
7         environmentalConditions.value[0].value = response.data.temperature
8         setStatus()
9       } catch (error) {
10        console.error('Error fetching data:', error)
11      }
12  }
13
14  setInterval(getEnvironmentalCondition, 60000)
```

Abbildung 100: getEnvironmentalCondition

Nach erfolgtem Einlesen der Umweltbedingungen gilt es, den Status der Pflanzen zu ermitteln und in die Kategorien "Ideal", "Akzeptabel" und "Mangelhaft" zu klassifizieren. Sollte sich mindestens eine Pflanze im mangelhaften Zustand befinden, wird eine *Message* generiert, um auf diese Umstände aufmerksam zu machen.

```
<Message v-if="deficientPlant" severity="error" :closable="false">
      <div class="text-sm"> {{ $t('deficientPlant') }} </div>
</Message>

function setStatus() {
  deficientPlant.value = false

  plantsByUser.value.forEach((plant) => {
    const matchingCondition = environmentalConditionByPlant.value.find(
         (condition) => condition.plantID === plant.ID)
    const lightIntensity = environmentalConditions.value[2].value
    const humidity = environmentalConditions.value[1].value
    const temperature = environmentalConditions.value[0].value

    let status
    if (
      humidity >= matchingCondition.idealHumMin &&
      humidity <= matchingCondition.idealHumMax &&
      lightIntensity >= matchingCondition.idealLightMin &&
      lightIntensity <= matchingCondition.idealLightMax &&
      temperature >= matchingCondition.idealTempMin &&
      temperature <= matchingCondition.idealTempMax
    ) status = "Ideal"
    else if (
      humidity >= matchingCondition.acceptHumMin &&
      humidity <= matchingCondition.acceptHumMax &&
      lightIntensity >= matchingCondition.acceptLightMin &&
      lightIntensity <= matchingCondition.acceptLightMax &&
      temperature >= matchingCondition.acceptTempMin &&
      temperature <= matchingCondition.acceptTempMax
    ) status = "Acceptable"
    else {
      status = "Deficient"
      deficientPlant.value = true
    }

    plant.status = status
  })
}
```

Abbildung 101: setStatus

https://github.com/Serifi/smart-urban-gardening-assistant

4. Reflexion

4.1. Meilenstein 1

4.1.1. Argjent Serifi

Der erste Meilenstein bildet eine solide Basis für das kommende Projekt. Die Webentwicklung ohne ein tiefgehendes theoretisches Verständnis mag anfangs möglich erscheinen, jedoch stellt sich schnell die Komplexität heraus, insbesondere wenn Fehler auftreten und deren Zusammenhänge aufgrund eines unzureichenden Verständnisses der Hintergrundaktivitäten nicht vollumfänglich ergründet werden können. Ein exemplarisches Szenario hierfür veranschaulicht sich im Lebenszyklus, wo Daten zwar augenscheinlich definiert werden, jedoch undefiniert präsentiert werden. Die Fehlermeldung gibt keine Hinweise darauf, dass die Abfolge der Hooks möglicherweise inkorrekt sein könnte, sondern hinweist nur darauf, dass die undefiniert erscheinen. Die potenziellen Ursachen für diesen Fehler muss man mit theoretischem Wissen begründen. Die eingeleitete praktische Ausarbeitung erwies sich als essenziell, um die reibungslose Funktionalität zu überprüfen, insbesondere weil niemand im Projektteam zuvor Erfahrungen in dieser speziellen Konstellation gesammelt hatte. Dabei wurde nicht nur die Funktionsfähigkeit des Arduino bestätigt, sondern es wurde auch bemerkt, dass ein Sensor fehlte, welcher jedoch für den zweiten Meilenstein rechtzeitig beschafft werden kann. Folglich verlief alles gemäß dem Plan, und die Kompetenzen in diesem Fachgebiet wurden erweitert, nicht zuletzt aufgrund der erstmaligen Einrichtung eines Vue-Projekts.

4.1.2. Natalia Chifa

Im Rahmen dieses Meilensteins manifestierte sich im Team eine konsequent positive Dynamik. Mittels einer präzisen Zuweisung der Aufgaben gelang es, den Fortschritt gezielt voranzutreiben. Hervorzuheben gelten insbesondere die offene Kommunikation sowie gegenseitige Unterstützung innerhalb des Teams. Anregungen an die Mitglieder, bei Herausforderungen ihre Hilfe anzubieten, trugen bedeutsam zur Kultivierung einer Atmosphäre der Zusammenarbeit bei. Darüber hinaus hat die Auseinandersetzung mit theoretischen Konzepten des Backends und der Datenkommunikation nicht nur einen tieferen Einblick in die Funktionsweise des zu implementierenden Systems ermöglicht, sondern auch die Fähigkeit, komplexe Zusammenhänge zu verstehen, gefördert.

4.1.3. Valon Hyseni

Innerhalb des ersten Meilensteins, welcher hauptsächlich der theoretischen Ausarbeitung des Embedded Systems gewidmet war, wurden erhebliche Fortschritte erzielt. Der Meilenstein bildet das Fundament für aufzubauendes Wissen in diesem Fachgebiet und bietet inspirierende Motivation. Das gewonnene Verständnis für die Zusammenarbeit zwischen Mikrocontroller und Sensoren ist bedeutsam für die spätere Integration in die Webanwendung.

4.2. Meilenstein 2

4.2.1. Argjent Serifi

Dieser Meilenstein nimmt eine wichtige Stellung des Projekts ein, indem er die Realisierung des bedeutendsten Aspekts, nämlich des Frontends der Webanwendung, in den Mittelpunkt rückt. Die grundlegenden Funktionalitäten wurden schnell implementiert, jedoch beanspruchten Feinheiten einen Hauptanteil der benötigten Zeit. Es war trotzdem wichtig, weil gerade diese kleinen, aber essenziellen Details von gleichrangiger Bedeutung sein sollten, um ein abgerundetes System von zu gestalten. In der Anfangsphase erfolgte die Programmierung eher improvisiert, um einen grundlegenden Rahmen zu schaffen. Mit fortschreitender Zeit erfolgte jedoch eine Auslagerung von Komponenten und die schrittweise Integration zusätzlicher Funktionalitäten. Obwohl der Code zwischenzeitlich unübersichtlich erschien, kristallisierte sich gegen Ende eine klare und kompakte Strukturierung in Form eigenständiger Komponenten heraus. Die Umsetzung erforderte ausgiebige Recherchen, experimentelle Herangehensweisen, kreativen Einsatz sowie umfassendes Testen, bis sämtliche Elemente harmonisch ineinandergriffen. Das Frontend der Webanwendung ist somit nahezu abgeschlossen, lediglich die Sprachauswahl und das Benutzerprofil stehen noch aus. Diese werden im kommenden Meilenstein ergänzt. Infolgedessen wurden die Kompetenzen im Umgang mit neuen Technologien erweitert, insbesondere im Kontext von Vue, welches durch eine schnelle Lernkurve überzeugt und zweifellos angenehmer ist als die herkömmliche Webentwicklung ohne Framework. Eine einzige Herausforderung besteht darin, dass Daten stets über Emits an die übergeordnete Komponente übermittelt werden müssen, weil eine Referenz über Properties aufgrund ihrer schreibgeschützten Beschaffenheit keine Änderungen zulässt. Anfangs war die genaue Implementierung noch recht offen, jedoch wurde das beste Verhältnis von Zeit und Leistung geschaffen, weil man im Frontend stets präziser, besser und umfangreicher entwickeln kann, man jedoch zu einem gewissen Zeitpunkt einen Schlussstrich ziehen muss.

4.2.2. Natalia Chifa

In dieser Phase war meine Hauptaufgabe, den Fortschritt genau zu beobachten, aus den Erfahrungen der anderen Mitglieder zu lernen und wichtige Erkenntnisse zu sammeln. Meine aktive Rolle war bislang eher begrenzt, jedoch freue ich mich auf die Möglichkeit, im nächsten Meilenstein meine Fähigkeiten im Backend zu erweitern. Diese neuen Erfahrungen werden es mir ermöglichen, einen bedeutenden Beitrag zu leisten und das Team effizient zu unterstützen. Die bisherige Arbeit im Frontend hat mir bereits wertvolle Einblicke gegeben, und ich bin zuversichtlich, dass meine zukünftige Beteiligung im Backend einen zusätzlichen Impuls für das Projekt darstellen kann.

4.2.3. Valon Hyseni

Im Fokus des zweiten Meilensteins stand die Initiation, ausgehend von der Konzeptionierung von Testdaten, welche eine essenzielle Rolle bei der Simulation des Systems spielen und zur Validierung beitragen. Parallel dazu wurde intensiv an der Entwicklung des ER-Modells gearbeitet, um die Grundlage für die Strukturierung der Datenbank im Rahmen des dritten Meilensteins zu schaffen. Darüber hinaus wurde mit der Herstellung der Verbindung zwischen dem Mikrocontroller und den verbleibenden Sensoren experimentiert. Infolge dieses Prozesses traten Herausforderungen auf, weil ein Sensor verspätet geliefert wurde und die Pins nicht gelötet waren. Dies führte dazu, dass nicht ausreichend Zeit zur Verfügung stand, um das Embedded System rechtzeitig fertigzustellen. Die wenig aussagekräftigen Fehlermeldungen trugen zusätzlich nicht zur Problembehebung bei, weil eine Vielzahl von Schnittstellen involviert war, sei es der USB-Anschluss, die Verkabelung oder Sensoren und vieles mehr. Dementsprechend verlief nicht alles nach Meilensteinplan, wodurch die verbleibende Projektzeit nunmehr dem Embedded System und Integration des Backends gewidmet wird.

4.3. Meilenstein 3

4.3.1. Argjent Serifi

Bei diesem letzten Meilenstein habe ich mich darauf beschränkt, geringfügige Ergänzungen im Frontend vorzunehmen und etwaige Fehler im Backend zu korrigieren. Anderweitig haben die Mitglieder des Teams ihre jeweiligen Aufgabenbereiche erfolgreich umgesetzt. Es war für mich erfreulich zu beobachten, dass alle mit Ehrgeiz bestrebt waren, ihre Probleme eigenständig zu bewältigen, ohne dabei sofort auf Unterstützung angewiesen zu sein. Die Webanwendung hat sich mindestens so erfolgreich entwickelt, wie es im ursprünglichen Plan vorgesehen war.

4.3.2. Natalia Chifa

In diesem Meilenstein stand die Entwicklung eines Backends im Fokus, das von einer vorherigen Erfahrung mit Flask umgestellt wurde. Diese Umstellung bedeutete eine erhebliche Lernherausforderung. Die Serverstruktur folgte einer typischen REST-API-Konfiguration mit klarer Trennung von Konfiguration, Datenbankverwaltung und Client. Technologien wie Node.js mit Express.js und MySQL-Client wurden verwendet. Wichtige Elemente des Server-Verzeichnisses waren node_modules, src, config.js und database.js. Die Datenbankstruktur umfasste Tabellen wie User, UserPlant, Plant und EnvironmentalCondition, die komplexen Beziehungen darstellten. Die Integration des Frontends mit dem Backend erfolgte durch Axios in einer Vue.js-Anwendung, wobei die Teamkollegen entscheidende Unterstützung leisteten. Die Anpassung an neue Technologien wie Vue.js, Node.js Express.js, MySQL erforderte Flexibilität und die Bereitschaft, sich neuen Herausforderungen zu stellen. Diese Erfahrung hat nicht nur das Verständnis für Backend-Entwicklung vertieft, sondern auch die Bedeutung von Teamarbeit und gegenseitiger Unterstützung

hervorgehoben. Die erfolgreiche Integration des Backends mit dem Frontend demonstrierte die Effektivität einer gut koordinierten Zusammenarbeit und legte den Grundstein für zukünftige Projekte, in denen ähnliche technologische Herausforderungen und Teamdynamiken zu erwarten sind. Trotz anfänglicher Schwierigkeiten führte die erfolgreiche Umsetzung zu einem funktionierenden Backend und einem erweiterten Verständnis für Backend-Entwicklung in verschiedenen Technologien.

4.3.3. Valon Hyseni

Im Rahmen des dritten Meilensteins habe ich das Embedded-System erfolgreich implementiert, wobei eine sorgfältige Verdrahtung der Sensoren vorgenommen und umfangreiche Recherchen durchgeführt wurden. Während des Prozesses wurde festgestellt, dass der ESP8266 Chip eine zusätzliche Komponente erfordert, um die Energieversorgung vom Arduino-Board adäquat zu regulieren. Diese Erkenntnis resultierte aus der Notwendigkeit, Überhitzung und mögliche Kurzschlüsse zu vermeiden. Bedauerlicherweise konnte das Embedded System infolgedessen nicht vollständig abgeschlossen werden, obgleich der Code bereits provisorisch vorhanden ist und lediglich die physische Verbindung aussteht. Parallel dazu habe ich intensiv am Backend gearbeitet, wobei insbesondere Unterstützung bei der Fehlerbehebung und dem Verständnis des Backends für eine Teamkollegin im Fokus standen. In retrospektiver Betrachtung ist die erbrachte Teamleistung zufriedenstellend, und bis auf unvorhergesehene Einflussfaktoren verlief der bisherige Verlauf positiv. Die noch ausstehenden Aufgaben werden bis zur finalen Abgabe termingerecht vervollständigt.

4.4. Endabgabe

Mit dem gebührenden Abschluss des Projekts möchten wir eine gemeinsame Reflexion verfassen, um unsere einheitliche Meinung zu unterstreichen. Es ist uns ein Anliegen zuerst unsere beträchtlichen Erkenntnisse hervorzuheben, welche wir im Zuge der eigenverantwortlichen Durchführung gewonnen haben. Es gelang uns innerhalb kurzer Zeit ein funktionierendes System aus vielfältigen Komponenten zu entwerfen und zu integrieren, dessen Anwendungsbereich von der hardwarenahen Programmierung bis zur Implementierung einer Webanwendung reichte. Infolgedessen war für sämtliche Teammitglieder ein bisher unbekannter Anwendungsbereich inkludiert, um somit gewiss neue Erkenntnisse zu erlangen. Im Kontext unserer Teammitglieder sollte das Team zum Abschluss erwähnt werden. Die Aufgabenverteilung und Kommunikation verlief reibungslos, sei es durch eigenständige Bearbeitung oder kooperativen Austausch zur gemeinsamen Lösungsfindung. Diese harmonische Zusammenarbeit trug dazu bei, einander optimal zu unterstützen und die vielfältigen Stärken gewinnbringend zu ergänzen. Das Projekt erweist sich zweifelsfrei als Erfolg und entspricht genau den zu Beginn des Semesters gehegten Erwartungen.

LITERATURVERZEICHNIS

[1] https://www.geeksforgeeks.org/introduction-to-internet-of-things-iot-set-1/ (03.11.2023)

[2] https://www.particle.io/iot-guides-and-resources/iot-protocols-and-standards/ (03.11.2023)

[3] https://www.interviewbit.com/blog/iot-architecture/ (03.11.2023)

[4] https://store.arduino.cc/products/arduino-uno-rev3 (01.11.2023)

[5] https://pixabay.com/de/illustrations/arduino-arduino-uno-technologie-2168193/ (01.11.2023)

[6] https://arduino-projekte.info/arduino-uno/ (01.11.2023)

[7] https://www.arduino.cc/reference/de/language/structure/sketch/setup/ (01.11.2023)

[8] https://www.arduino.cc/reference/de/language/structure/sketch/loop/ (01.11.2023)

[9] https://wiki.hshl.de/wiki/index.php/Temperatur-_und_Feuchtigkeitssensor_DHT11 (01.11.2023)

[10] https://www.studiopieters.nl/bh1750-light-sensor-module/ (01.11.2023)

[11] https://www.az-delivery.de/collections/esp8266 (01.11.2023)

[12] https://www3.tuhh.de/itbh/informatik-
202021/veranstaltungsskript/python/processingpy/einleitung/was-ist-esp8266/ (01.11.2023)

[13] https://forum.fritzing.org/t/esp8266-nodemcu-lolin/6386 (01.11.2023)

[14] https://de.wikipedia.org/wiki/Hypertext_Transfer_Protocol (03.11.2023)

[15] https://www.ionos.de/digitalguide/hosting/hosting-technik/http-request-erklaert/ (03.11.2023)

[16] https://vuejs.org/guide/essentials/component-basics.html (01.11.2023)

[17] https://ambient.digital/wissen/ambipedia/vue-js/ (01.11.2023)

[18] https://medium.com/js-dojo/exploring-vue-js-reactive-two-way-
data-binding-da533d0c4554 (01.11.2023)

[19] https://vuejs.org/guide/scaling-up/sfc.html (01.11.2023)

[20] https://vuejs.org/guide/scaling-up/sfc.html#what-about-separation-of-concerns (01.11.2023)

[21] https://www.tatvasoft.com/outsourcing/2021/10/what-
is-vue-js-and-its-benefits.html (01.11.2023)

[22] https://vuejs.org/guide/extras/rendering-mechanism.html (01.11.2023)

[23] https://kinsta.com/de/wissensdatenbank/nuxt-js/ (01.11.2023)

[24] https://de.wikipedia.org/wiki/MySQL (01.11.2023)

[25] https://www.talend.com/de/resources/was-ist-mysql/ (01.11.2023)

[26] https://www.ionos.de/digitalguide/server/sicherheit/sql-injection-grundlagen-und-
schutzmassnahmen/ (01.11.2023)

[27] https://builtin.com/data-science/data-validation (01.11.2023)

[28] https://www.avenga.com/magazine/data-validation-testing/ (01.11.2023)

ABBILDUNGSVERZEICHNIS

Abbildung 1: IoT-Architektur 10
Abbildung 2: Arduino 12
Abbildung 3: Grundgerüst eines Programms in Arduino 13
Abbildung 4: ESP8266 [13] 14
Abbildung 5: BH1750 [10] 14
Abbildung 6: DHT11 [9] 14
Abbildung 7: Komponentenbaum 16
Abbildung 8 : Datenbindung mittels Direktiven 16
Abbildung 9: Message.vue 17
Abbildung 10: Render Pipeline 17
Abbildung 11: Verzeichnisstruktur 17
Abbildung 12: Tailwind CSS inline styling 18
Abbildung 13: Flex im responsiven Design 18
Abbildung 14: Verkabelung des DHT11 20
Abbildung 15: Programm des DHT11 20
Abbildung 16: Projektinitialisierung 21
Abbildung 17: Projektstart 21
Abbildung 18: nuxt.conig.ts 21
Abbildung 19: plugin/index.js 21
Abbildung 20: Anzeige des eingerichteten Projekts 22
Abbildung 21: Login am Desktop 23
Abbildung 23: Anmeldung am Smartphone 23
Abbildung 22: Registrierung am Smartphone 23
Abbildung 24: tailwind.config.ts 24
Abbildung 25: Card.vue 24
Abbildung 26: Beschreibung der Startseite 25
Abbildung 27: Benutzername und Passwort der Startseite 25
Abbildung 28: Link, Button und Meldung der Startseite 25
Abbildung 29: Meldung bei fehlerhaften Eingabedaten der Startseite 26
Abbildung 30: Variablen des Logins 26
Abbildung 31: Funktionen des Logins 26
Abbildung 32: Style des Logins 26
Abbildung 33: Empfehlungen eines sicheren Passworts 27
Abbildung 34: Erweiterung von Password zur Registrierung 27

Abbildung 35: Funktion zur Registerierung 27
Abbildung 36: Template der Startseite 2
Abbildung 37: Skript der Startseite 2
Abbildung 38: Hauptseite 29
Abbildung 39: Tabletansicht der Hauptseite 29
Abbildung 40: Smartphoneansicht der Hauptseite 29
Abbildung 41: Titel der Hauptseite 29
Abbildung 42: Pflanzen suchen in der Hauptseite 2
Abbildung 43: Pflanze hinzufügen in der Hauptseite 2
Abbildung 44: Pflanze hinzufügen in der Hauptseite 2
Abbildung 45: Sprachauswahl und Benutzerprofil der Hauptseite 2
Abbildung 46: Sprachauswahl der Hauptseite 2
Abbildung 47: Sprachauswahl der Hauptseite 2
Abbildung 48: Benutzerprofil der Hauptseite 2
Abbildung 49: Benutzerprofil der Hauptseite 2
Abbildung 50: Variablen des Headers 2
Abbildung 51: Funktionen des Headers 2
Abbildung 52: Beschreibung der Hauptseite 2
Abbildung 53: Filter der Hauptseite 2
Abbildung 54: Umweltbedingungen der Hauptseite 2
Abbildung 55: Variablen des Filters 2
Abbildung 56: Funktionen des Filters 2
Abbildung 57: Header und Filter in Hauptseite 2
Abbildung 58: Pflanzentabelle der Hauptseite 2
Abbildung 59: Pflanzentabelle der Hauptseite 2
Abbildung 60: Status und Pflanze löschen in der Hauptseite 2
Abbildung 61: Pflanzentabelle der Hauptseite 2
Abbildung 62: Variablen der Hauptseite 2
Abbildung 63: Gefilterte Pflanzen der Hauptseite 2
Abbildung 64: Hinzufügen und Entfernen von Pflanzen in der Hauptseite 38
Abbildung 65: Datenmodell 38
Abbildung 66: Aktivierung des Buttons bei Eingabe 41
Abbildung 67: stores/index.js 41
Abbildung 68: stores in plugins/index.js 41
Abbildung 69: Benutzer persistieren 40
Abbildung 70: Benutzer erhalten 40

Abbildung 71: locales 40
Abbildung 72: Konfiguration von vue-i18n 40
Abbildung 73: Sprachauswahl 40
Abbildung 74: Anwendung von locales 40
Abbildung 75: Benutzereinstellungen 40
Abbildung 76: Benutzereinstellungen 40
Abbildung 77: Arduino Library Manager 40
Abbildung 78: Verkabelung des BH1750 40
Abbildung 79: Programm des BH1750 40
Abbildung 80: Verzeichnisstruktur des Servers 40
Abbildung 81: Datenbankzugriff über SQLTools 40
Abbildung 82: config.js 41
Abbildung 83: database.js 41
Abbildung 84: app.js 41
Abbildung 85: Ausschnitt aus initDatabase.sql 40
Abbildung 86: getUsers Endpoint 40
Abbildung 87: getUsers Implementierung 40
Abbildung 88: getUserByName Endpoint 40
Abbildung 89: checkUserExists Implementierung 40
Abbildung 90: register Endpoint 40
Abbildung 91: putPassword Endpoint 40
Abbildung 92: updatePassword Implementierung 40
Abbildung 93: deleteUserPlant Endpoint 40
Abbildung 94: confirmDeletion Implementierung 40
Abbildung 95: deleteUser Endpoint 40
Abbildung 96: Verkabelung des ESP8266 40
Abbildung 97: Setup der Kommunikation 40
Abbildung 98: Loop der Kommunikation 40
Abbildung 99: Funktionen der Kommunikation 39
Abbildung 100: getEnvironmentalCondition 39
Abbildung 101: setStatus 40

TABELLENVERZEICHNIS

Tabelle 1: Beispiel anhand einer Tomate (ChatGPT, 2023) 5
Tabelle 2: Meilenstein #1 8
Tabelle 3: Meilenstein #2 8
Tabelle 4: Meilenstein #3 8
Tabelle 5: Datenblatt des Arduinos 12
Tabelle 6: Datenblatt des DHT11 13
Tabelle 7: HTTP-Methoden 15
Tabelle 8: HTTP-Statuscodes 15